

New Scalable Decoder Architectures for Reed–Solomon Codes

Yingquan Wu, *Senior Member, IEEE*

Abstract—In this paper, we devise new scalable decoder architectures for Reed–Solomon (RS) codes, comprising three parts: error-only decoding, error-erasure decoding, and their decoding for singly extended RS codes. New error-only decoders are devised through algorithmic transformations of the inversionless Berlekamp–Massey algorithm (IBMA). We first generalize the Horiguchi–Koetter formula to evaluate error magnitudes using the error locator polynomial $\Lambda(x)$ and the auxiliary polynomial $B(x)$ produced by IBMA, which effectively eliminates the computation of error evaluator polynomial. We next devise an enhanced parallel inversionless Berlekamp–Massey algorithm (ePIBMA) that effectively takes advantage of the generalized Horiguchi–Koetter formula. The derivative ePIBMA architecture requires only $2t + 1$ (t denotes the error correction capability) systolic cells, in contrast with $3t$ or more cells of the existing regular architectures based on IBMA or the Euclidean algorithm. Moreover, it may literally function as a linear-feedback-shift-register encoder. New error-erasure decoders are devised through algorithmic transformations of the inversionless Blahut algorithm (IBA). The proposed split parallel inversionless Blahut algorithm (SPIBA) yields merely $2t + 1$ systolic cells, which is the same number as the error-only decoder ePIBMA. The task is partitioned into two separate steps, computing the complementary error-erasure evaluator polynomial followed by computing error-erasure locator polynomial, both utilizing SPIBA. Surprisingly, it has exactly the same number of cells and literally the same complexity and throughput as the proposed error-only decoder architecture ePIBMA; it employs 33% less hardware and at the same time achieves more than twice faster throughput, than the serial architecture IBA. We further propose a unified parallel inversionless Blahut algorithm (UPIBA) by incorporating the key virtues of the error-only decoder ePIBMA into SPIBA. The complexity and throughput of the derivative UPIBA architecture are literally the same as ePIBMA and SPIBA, while performing almost equally efficiently as ePIBMA on error-only decoding and as SPIBA on error-erasure decoding. UPIBA also inherits the dynamic power saving feature of ePIBMA and SPIBA. Indeed, UPIBA renders highly attractive for on-the-fly implementation of error-erasure decoding. We finally demonstrate that the proposed decoders, i.e., ePIBMA, SPIBA, and UPIBA, can be magically migrated to decode singly extended RS codes, with negligible add-ons, except that an extra multiplexer is added to their critical paths. To the author’s best knowledge, this is the first time that a high-throughput decoder for singly extended RS codes is explored.

Index Terms—Reed–Solomon codes, singly extended Reed–Solomon codes, inversionless Berlekamp–Massey algorithm, inversionless Blahut algorithm, Horiguchi–Koetter formula, high-throughput decoder architecture, unified decoder.

I. INTRODUCTION

REED–SOLOMON (RS) codes are a class of cyclic codes featured as maximal distance separable. Specifically, a Reed–Solomon (n, k) code over a finite field $\text{GF}(q)$ satisfies $n < q$ and achieves the maximally separable distance $d = n - k + 1$. RS codes play important roles in modern communication systems and data storage systems, such as power line communications, digital video broadcasting terrestrial system, vestigial sideband system, cable modem, optical communications, DVD, and DRAM, etc. Their decoder architecture has been extensively explored and remains a challenging research topic.

There are three different decoding algorithms to correct up to $t \triangleq \lfloor \frac{d-1}{2} \rfloor$ errors with complexity $O(dn)$, namely, the Berlekamp–Massey algorithm [1], the extended Euclidean algorithm [2], and the Welch–Berlekamp algorithm [3]. However, the last one is largely neglected by VLSI researcher community due to its algorithmic irregularity and high cost of on-the-fly polynomial evaluation. Initially, the extended Euclidean algorithm received popularity in VLSI community and industry practice, due to its natural regularity and short critical path delay [4]–[9]. Arguably, Shao *et al.* [4] devised the first full-throughput high-speed implementation of the extended Euclidean algorithm, which uses $4t + 4$ cells whose complexity is dominated by 2 multipliers, and has a critical path delay of one multiplication, one addition, and one multiplex. In [5], Berlekamp *et al.* presented a folded architecture of the extended Euclidean algorithm, which uses $2t + 1$ multipliers but operates $12t$ cycles (against the full-parallelism of $2t$ cycles). Recently, Baek and Sunwoo [8] presented a degree computationless modified Euclid algorithm (DCMEA) which yields a systolic architecture utilizing only $3t + 2$ cells. In [9], the authors further reduce the number of cells to $3t$.

For long time, relatively few practical implementations utilize the Berlekamp–Massey algorithm, presumably because the architectures were found to be irregular and to have a longer critical path delay that was also dependent on the value of t . In [10], Reed *et al.* presented an inversionless Berlekamp–Massey algorithm and translated it straightforwardly into an architecture, whose critical path passes through two multipliers and $1 + \lceil \log_2(t + 1) \rceil$ adders. In [11], Sarwate and Shanbhag presented a high-speed systolic architecture, **RiBMA**, with $3t + 1$ cells, whose critical path passes through only one multiplier and one adder. Its high-speed is achieved by transforming

Manuscript received January 3, 2015; revised May 19, 2015 and June 9, 2015; accepted June 11, 2015. Date of publication June 15, 2015; date of current version August 7, 2015. The material is partially patented, under U.S. Patents No. 7 613 988 (Nov. 3, 2009), 7 716 562 (May 11, 2010), and 7 870 469 (Jan. 11, 2011). The associate editor coordinating the review of this paper and approving it for publication was K. Abdel-Ghaffar.

The author is with Tidal Systems Inc., Santa Clara, CA 95054 USA (e-mail: yingquan_wu@yahoo.com).

Digital Object Identifier 10.1109/TCOMM.2015.2445759

the algorithm such that the error locator polynomial and the complementary error evaluator polynomial are simultaneously updated, so that the discrepancy (which is simply a coefficient of the complementary error evaluator polynomial) computation is effectively avoided. This systolic architecture allows to be arbitrarily folded to trade off gate count and latency [12]. In [13], it was claimed to reduce the **RiBMA** architecture from $3t + 1$ cells to $2t + 2$ cells through smartly truncating unused cells. However, the work is invalid since it only considers a single case where the length of the error locator polynomial increases by 1 for every two iterations. For instance, when there is a single error, the concatenated auxiliary polynomial $\Theta(x)$ at the end of iterations has $3t$ valid terms, therefore, must be represented by at least $3t$ cells. The Horiguchi–Koetter formula [15, p. 225] is an alternative way to compute error magnitudes by directly exploiting the error locator polynomial in conjunction with its auxiliary polynomial generated by the Berlekamp–Massey algorithm, in contrast to the conventional Forney’s formula which requires to compute the error evaluator polynomial. Interestingly, the Horiguchi–Koetter formula was largely neglected by the VLSI researcher community, partially because the state-of-the-art architecture **RiBMA** computes both error locator polynomial and complementary error evaluator polynomial simultaneously, enabling a variant Forney formula for error evaluation.

There exist two different approaches to perform error-erasure decoding. Specifically, Forney’s approach first removes erasure information from syndromes and then applies error-only decoding [14], whereas, Blahut’s approach is to unify errors and erasures into a single algorithm, by initializing the error locator and auxiliary polynomials with the erasure locator polynomial in the Berlekamp–Massey algorithm [15]. Boyer [16] built a pipelined high-speed RS error-erasure decoder utilizing Forney’s approach in conjunction with the inversionless Berlekamp–Massey algorithm, which processes multiple codewords in a pipelined fashion. Oh and Oh [19] developed an error-erasure decoder architecture in light of Forney’s approach in conjunction with the extended Euclidean algorithm. Banks [20] improved the architecture in [19] by having a single polynomial time multiplexed to produce firstly a modified syndrome polynomial and then an erasure locator polynomial. Chang *et al.*, presented a universal architecture for Forney’s error-erasure decoders with configurable code rates and operation fields, based on Montgomery multiplication algorithm. Jeng *et al.* [17] extended the inverse-free property from the Berlekamp–Massey algorithm for error-only decoding to the Blahut algorithm for error-erasure decoding. In [22], the authors presented a reformulated inversionless Blahut algorithm (**rIBA**) in which the majority complexity carrier, the iterative update of error-erasure locator polynomial is homogeneous, regardless of errors or erasures. The derivative **rIBA** architecture employs $6t + 2$ multipliers and exhibits a critical path delay of two multiplications, $2 + \lceil \log_2 t \rceil$ XOR’s, and one multiplex. Truong *et al.* [22], [23] constructed a novel algorithm that employs the extended Euclidean algorithm to iteratively update error-erasure evaluator polynomial and produce discrepancy values and in parallel, performs **rIBA** to iteratively update the error-erasure locator polynomial. Its obvious advantages

are avoidance of the explicit computation of discrepancy and iterative computation of both polynomials simultaneously. It employs $4t + 4$ systolic cells and exhibits a critical path delay of one multiplication, one addition, and two multiplexes. Overall, decoder architectures based on Forney’s approach must perform three inhomogeneous functional blocks, namely, error-only syndrome computation by removing erasure syndromes, key-equation-solver to compute error locator polynomial, and computing error-erasure locator polynomial by adding roots of inverse erasure locators; whereas the architectures based on Blahut’s approach naturally compute the error-erasure locator polynomial within a functional block and thus are favored in practices.

The first part of the paper tackles error-only decoding of RS codes. We explore scalable decoder architectures through algorithmic reformulation toward IBMA. IBMA utilizes syndromes to iteratively computes the error locator polynomial, denoted by $\Lambda(x)$, and its auxiliary polynomial, denoted by $B(x)$. We first show that the effective length (by removing zeros) of $B(x)$ is at most $t - 1$ (t denotes error correction capability). We further show that the error evaluation using the generalized Horiguchi–Koetter formula, which directly employs $\Lambda(x)$ and $B(x)$, exhibits almost equal complexity as the conventional Forney formula, which employs $\Lambda(x)$ and error evaluator polynomial, denoted by $\Omega(x)$, while eliminating the extra computation for $\Omega(x)$. By enforcing a dynamic stopping rule to terminate early when the length of $B(x)$ reaches $t - 1$, we devise an enhanced Inversionless Berlekamp–Massey algorithm (**eIBMA**). As the core of the part, we reformulate **eIBMA** to an enhanced parallel inversionless Berlekamp–Massey algorithm (**ePIBMA**), by simultaneously computing discrepancy and updating error evaluator polynomial. The resulting systolic architecture **ePIBMA** (Throughout the paper, we use an acronym to represent an algorithm and the acronym in bold for the derivative architecture) exhibits the desirable critical path delay of one multiplication and one addition, and requires only $4t + 2$ multipliers and registers. This is significantly less than existing low-latency decoder architectures in the literature, and moreover, it is capable of dynamically disabling the multipliers and adders when not needed. In addition, it has another key feature to function as a linear-feedback-shift-register (LFSR) encoder. This allows to save the dedicated encoder when encoding and decoding are divided in time.

The second part concerns error-erasure decoding. We first devise a retimed parallel inversionless Blahut algorithm (**rPIBA**) by transforming **rIBA** presented in [22]. The resulting **rPIBA** architecture has $4t + 1$ systolic cells with $8t + 2$ multipliers and a critical path delay of one multiplication, one addition, and one multiplex. It iteratively computes the complementary error-erasure evaluator polynomial and error-erasure locator polynomial simultaneously in $2t$ cycles. The architecture appears similar to the one presented in [22]. However, the proposed algorithm iteratively computes the complement of the error-erasure evaluator polynomial, whereas the latter computes the error-erasure evaluator polynomial itself; the proposed algorithm utilizes $4t + 1$ systolic cells, whereas the one in [22] employs $4t + 4$ cells. In addition, we also incorporate a novel early stopping rule to save power dynamically when the errors

and erasures are (predominantly) well within the correction capability. By splitting the computation of the complementary error-erasure evaluator polynomial and error-erasure locator polynomial in **rPIBA**, we devise a split parallel inversionless Blahut algorithm (**SPIBA**) whose derivative architecture effectively reduces the number of cells to $2t + 1$ by doubling latency to $4t$ cycles. Surprisingly, it has exactly the same number of cells and almost the same hardware complexity and clock frequency as the proposed error-only decoder architecture **ePIBMA**; it employs 33% less hardware, at the same time achieves more than twice faster throughput, than the serial architecture **rIBA**. Finally, we propose a unified parallel inversionless Blahut algorithm (**UPIBA**) by incorporating the key virtues of the error-only decoder **ePIBMA** into **SPIBA**. Its derivative architecture **UPIBA** employs $2t + 1$ systolic cells, with complexity and throughput literally same as **ePIBMA** and **SPIBA**, while performing error-only decoding nearly at the efficiency of the dedicated **ePIBMA** and error-erasure decoding nearly at the efficiency of the dedicated **SPIBA**. **UPIBA** also inherits the dynamic power saving feature from both **ePIBMA** and **SPIBA**. It is worth noting that all proposed parallel algorithms yield systolic architectures and thus can be arbitrarily folded to trade off hardware complexity and latency.

The third part explores decoding singly extended RS (SERS) codes. Primitive RS codes in $\text{GF}(2^m)$ carry an undesirable property that the code length $n = 2^m - 1$ is an odd number, whereas a practical system largely has the preference of transmitting even number of symbols. For this reason, singly extended RS (SERS) codes are actually employed in a number of applications, e.g., the (128, 122) SERS code is used as part of forward-error-correction in Cable TV [25]. In [26], SERS codes are modified to construct a new family of 2-D codes for asynchronous optical code-division multiple access. An error-only decoding algorithm by slightly modifying the Berlekamp–Massey algorithm is given in [15]. The error-erasure decoding is incorrectly addressed in [27], noting it does not consider the scenario that the extended symbol is erroneous or erased. We demonstrate that all proposed systolic decoder architectures, namely, **ePIBMA**, **SPIBA**, and **UPIBA**, are readily deployed to decode SERS codes with little add-on. The only noticeable degradation is that an extra multiplexer is added to the critical paths of all three decoders.

The rest of the paper is organized as follows. Section II introduces RS codes in hardware perspectives, including LFSR encoding, the inversionless Berlekamp–Massey algorithm for error-only decoding, and, the inversionless Blahut algorithm as well as its hardware amenable reformulation for error-erasure decoding. Section III presents an enhanced inversionless Berlekamp–Massey algorithm by incorporating a dynamic early stopping rule. Section IV devises a parallel inversionless Berlekamp–Massey algorithm and its enhanced systolic architecture, **ePIBMA**, which significantly outperform the existing ones in literature. In Section V, we first present a parallel inversionless Blahut algorithm and its retimed systolic architecture **rPIBA**. We then halve **rPIBA** to form a new architecture **SPIBA** which doubles the number of cycles. In Section VI, we inherently merge **SPIBA** and the error-only decoder **ePIBMA** into a unified architecture, **UPIBA**, which literally retains the

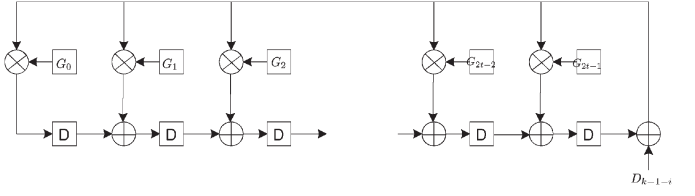


Fig. 1. The LFSR encoder architecture.

efficiency of **SPIBA** for error-erasure decoding and **ePIBMA** for error-only decoding. In Section VII, we make minor modifications on **ePIBMA** and **SPIBMA** to decode SERS codes for the cases of error-only and error-erasure, respectively. Section VIII concludes with pertinent remarks.

II. PRELIMINARIES

A. Reed–Solomon Codes

For a Reed–Solomon $\mathcal{C}(n, k)$ code over $\text{GF}(p^m)$, a k -symbol dataword $\mathbf{D} \triangleq [D_{k-1}, D_{k-2}, \dots, D_1, D_0]$ is encoded to an n -symbol codeword $\mathbf{C} \triangleq [C_{n-1}, C_{n-2}, \dots, C_1, C_0]$, or more conveniently, a dataword polynomial $D(x) = D_{k-1}x^{k-1} + D_{k-2}x^{k-2} + \dots + D_1x^1 + D_0$ is encoded to a codeword polynomial $C(x) = C_{n-1}x^{n-1} + C_{n-2}x^{n-2} + \dots + C_1x + C_0$, by means of a generator polynomial

$$G(x) \triangleq \prod_{i=0}^{n-k-1} (x - \alpha^{m_0+i}) \\ = x^{n-k} + G_{n-k-1}x^{n-k-1} + \dots + G_1x + G_0,$$

where m_0 is typically 0 or 1 and α is a primitive element of $\text{GF}(p^m)$ (in this presentation we do not distinguish between a vector $\mathbf{A} = [A_l, A_{l-1}, \dots, A_1, A_0]$ and its polynomial representation $A(x) = A_lx^l + A_{l-1}x^{l-1} + \dots + A_1x + A_0$). A polynomial of degree less than n is a codeword polynomial if and only if it is a multiple of the generator polynomial $G(x)$. As can be readily seen, a codeword polynomial $C(x)$ satisfies

$$C(\alpha^{m_0+i}) = 0, \quad i = 0, 1, 2, \dots, n - k - 1.$$

The minimum Hamming distance of the code is $d_{\min} = n - k + 1$, a feature that makes the code known as *maximally-distance-separable* [15]. To simplify the presentation as well as to limit to the practical interests, we set $p = 2$ (i.e., $\text{GF}(2^m)$ instead of general cases $\text{GF}(p^m)$), $m_0 = 1$ (i.e., the code is defined in *narrow-sense*), and $n - k = 2t$ (where t is known as the *error-correction capability*); while it will be straightforward to extend our presentation to general cases.

A systematic encoding generates a codeword that is comprised of the dataword followed by parity-check symbols. This can be obtained as follows. Let $\Psi(x) \triangleq \Psi_{2t-1}x^{2t-1} + \Psi_{2t-2}x^{2t-2} + \dots + \Psi_1x + \Psi_0$ denote the remainder obtained by dividing $x^{2t}D(x)$ by $G(x)$. Then, the polynomial $x^{2t}D(x) - \Psi(x)$ is a multiple of $G(x)$ and is denoted as a systematic codeword polynomial; alternatively, $\mathbf{C} = [D_{k-1}, D_{k-2}, \dots, D_0, -\Psi_{2t-1}, -\Psi_{2t-2}, \dots, -\Psi_0]$. Encoder can be efficiently implemented by a linear-feedback-shift-register (LFSR) circuit, as

depicted in Fig. 1 [15, pp. 235–237]. It is worth emphasizing that the systematic codeword is reversely indexed in polynomial representation, i.e., $\mathbf{C} = [C_{n-1}, C_{n-2}, \dots, C_1, C_0]$. Therefore, an error location i corresponds to the error locator α^{n-1-i} , conversely, an error locator α^j points to the error location $n - 1 - j$.

B. Berlekamp–Massey Algorithm for Error-Only Decoding

Let $C(x)$ denote the transmitted codeword polynomial and $R(x)$ the received word polynomial after appropriate channel quantization. We next briefly introduce the prevalent decoding methodology, which lays down the foundation of our architectures. It begins by computing syndrome values

$$S_i = R(\alpha^{i+1}), \quad i = 0, 1, 2, \dots, 2t - 1.$$

If all $2t$ syndrome values are zero, then $R(x)$ is a codeword and thus is presumed that $C(x) = R(x)$, i.e., no errors have occurred. Otherwise, the decoder attempts to solve the following equation system

$$\begin{cases} Y_1 X_1^1 + Y_2 X_2^1 + \dots + Y_e X_e^1 = S_0 \\ Y_1 X_1^2 + Y_2 X_2^2 + \dots + Y_e X_e^2 = S_1 \\ \vdots \\ Y_1 X_1^{2t} + Y_2 X_2^{2t} + \dots + Y_e X_e^{2t} = S_{2t-1} \end{cases}$$

where e denotes the (unknown) number of errors, $X_1, X_2, \dots, X_e$ denote the error locators, and $Y_1, Y_2, \dots, Y_e$ denote the corresponding error magnitudes.

Define the *syndrome* polynomial

$$S(x) \triangleq S_0 + S_1 x + S_2 x^2 + \dots + S_{2t-1} x^{2t-1}. \quad (1)$$

Define the *error locator* polynomial

$$\Lambda(x) \triangleq \prod_{i=1}^e (1 - X_i x) = 1 + \Lambda_1 x + \Lambda_2 x^2 + \dots + \Lambda_e x^e \quad (2)$$

where all inverted error locators are the roots of $\Lambda(x)$. Define the *error evaluator* polynomial

$$\begin{aligned} \Omega(x) &\triangleq \sum_{i=1}^e Y_i X_i \prod_{j=1, j \neq i}^e (1 - X_j x) \\ &= \Omega_0 + \Omega_1 x + \Omega_2 x^2 + \dots + \Omega_{e-1} x^{e-1}. \end{aligned} \quad (3)$$

The three polynomials satisfy the following *key equation* [1], [15]

$$\Omega(x) = \Lambda(x)S(x) \pmod{x^{2t}}. \quad (4)$$

Both the Berlekamp–Massey (B–M) algorithm and the extended Euclidean algorithm can be used to solve the above key equation with $O(t^2)$ operations, given that the number of

errors e does not exceed the error-correction capability t . Our presentation will be focused on the inversionless B–M algorithm (IBMA), whose pseudo code is given below [10]:

Inversionless B–M Algorithm (IBMA)

- Input: $\mathbf{S} = [S_0, S_1, S_2, \dots, S_{2t-1}]$
 - Initialization: $\Lambda^{(0)}(x) = B^{(0)}(x) = 1$, $\gamma^{(0)} = 1$, $L_\Lambda^{(0)} = L_B^{(0)} = 0$
 - For $r = 0, 1, 2, \dots, 2t - 1$, do:
 - Compute $\Delta^{(r)} = \sum_{i=0}^{L_\Lambda^{(r)}} \Lambda_i^{(r)} \cdot S_{r-i}$
 - Compute $\Lambda^{(r+1)}(x) = \gamma^{(r)} \cdot \Lambda^{(r)}(x) - \Delta^{(r)} \cdot x B^{(r)}(x)$ (un-normalized)
 - If $\Delta^{(r)} \neq 0$ and $L_\Lambda^{(r)} \leq L_B^{(r)}$, then
 - * Set $B^{(r+1)}(x) \leftarrow \Lambda^{(r)}(x)$, $\gamma^{(r+1)} \leftarrow \Delta^{(r)}$ (un-normalized)
 - * Set $L_\Lambda^{(r+1)} \leftarrow L_B^{(r)} + 1$, $L_B^{(r+1)} \leftarrow L_\Lambda^{(r)}$
 - Else
 - * Set $B^{(r+1)}(x) \leftarrow x B^{(r)}(x)$, $\gamma^{(r+1)} \leftarrow \gamma^{(r)}$
 - * Set $L_\Lambda^{(r+1)} \leftarrow L_\Lambda^{(r)}$, $L_B^{(r+1)} \leftarrow L_B^{(r)} + 1$
 - Outputs: $\mathbf{\Lambda} = [\Lambda_0^{(2t)}, \Lambda_1^{(2t)}, \dots, \Lambda_t^{(2t)}]$, $\mathbf{B} = [B_0^{(2t)}, B_1^{(2t)}, \dots, B_{2t-1}^{(2t)}]$, $\gamma^{(2t)}$, $L_\Lambda^{(2t)}$
-

Note that in IBMA description, we used superscript “ (r) ” to stand for the r -th iteration and subscript “ i ” the i -th coefficient. Theoretically, it is rare but possible that an error locator polynomial $\Lambda(x)$ with length $L_\Lambda > t$ contains all error locators as the inverse of its roots. But, in practice a failure is claimed when $L_\Lambda > t$, this is because it is not worthwhile putting extra circuit. This is reflected in the above algorithmic output $\mathbf{\Lambda}$. Readers are referred to [1], [15] for algorithmic assertion and the logical meaning of the length of a LFSR.

It is worth noting for two different indexing conventions in the presentation. Specifically, encoding polynomials, including dataword, codeword, and generator polynomial, are indexed in descending order, which accommodates the nature of systematic encoding; whereas decoding polynomials, including syndrome, error locator, auxiliary, error evaluator, etc. polynomial, are indexed in ascending order, which accommodates the nature of the scalable error correction capability and indeterministic number of errors.

Once given the error locator polynomial $\Lambda(x)$, the error locators along with corresponding error locations are obtained through the Chien search (which will be discussed shortly). The corresponding error magnitudes can be obtained from the Forney formula [15, p. 196],

$$\begin{aligned} Y_i &= \frac{\Omega(X_i^{-1})}{\Lambda'(X_i^{-1})} \\ &= \frac{X_i^{-1} \Omega(X_i^{-1})}{\Lambda_{\text{odd}}(X_i^{-1})}, \quad i = 1, 2, \dots, e. \end{aligned} \quad (5)$$

where $\Lambda_{\text{odd}}(x)$ denotes the odd-term polynomial of $\Lambda(x)$, and the second equality is due to operating in binary extension field $\text{GF}(2^m)$, such that

$$\begin{aligned}\Lambda'(x) &= \Lambda_1 + 2\Lambda_2x + 3\Lambda_3x^2 + \dots + t\Lambda_tx^{t-1} \\ &= \Lambda_1 + \Lambda_3x^2 + \Lambda_5x^4 + \dots + \Lambda_{t+\delta_t-1}x^{t+\delta_t-2} \\ &= x^{-1}\Lambda_{\text{odd}}(x)\end{aligned}$$

where δ_i is set to 1 if i is odd or 0 otherwise. $\Lambda_{\text{odd}}(X_i^{-1})$ is handily computed inside the Chien search circuit, which is commonly employed to determine the roots of $\Lambda(x)$ [24]. The Chien search circuit evaluates $\Lambda(x)$ in the sequential order of $1, \alpha^{-1}, \alpha^{-2}, \dots, \alpha^{-(n-1)}$, utilizing an array of t half-multipliers associated with constants, $\alpha^{-1}, \alpha^{-2}, \dots, \alpha^{-t}$, respectively [24]. Recall that LFSR encoding yields reverse indexing, i.e., $\mathbf{C} = [C_{n-1}, C_{n-2}, \dots, C_1, C_0]$. We clarify that, when α^{-j} is a root of $\Lambda(x)$, α^j is the corresponding error locator whereas $n-1-j$ is the corresponding error location. It is worth pointing out that the proper root search sequence is $1, \alpha^{-1}, \alpha^{-2}, \dots, \alpha^{-(n-1)}$. For majority applications of shortened codes, this sequence is different from the sequence $1, \alpha^1, \alpha^2, \dots, \alpha^{n-1}$, or the sequence $\alpha^1, \alpha^2, \dots, \alpha^n$ (which was incorrectly investigated in [28], [29]).

In practice, the decoding is commonly pipelined in three stages, syndrome computation (SC) block, key-equation solver (KES) block, Chien search and error evaluation (CSEE) block, respectively. Since the SC and CSEE blocks both take n (the length of code) cycles, whereas the KES takes only $2t$ cycles which is typically much smaller than n , a common-practice folds KES circuit (by $\lceil \frac{n}{2t} \rceil$ times, if systolic) to reduce the circuit complexity while equalizing the latencies of three pipelining stages.

C. Blahut Algorithm for Error-Erasure Decoding

Assume there are ρ erasure locators $\alpha_0, \alpha_1, \dots, \alpha_{\rho-1}$, respectively. Error-erasure decoding succeeds if the number of errors, denoted by f and the number of erasures ρ , satisfy

$$2f + \rho \leq 2t.$$

The Blahut algorithm treats errors and erasures in a unified manner and thus results in great hardware amenity. The following pseudo code of the Inversionless Blahut Algorithm (IBA) is due to [17]:

Inversionless Blahut Algorithm (IBA)

- Inputs: $\mathbf{S} = [S_0, S_1, S_2, \dots, S_{2t-1}]$, $[\alpha_0, \alpha_1, \dots, \alpha_{\rho-1}]$
- Initialization: $\Lambda^{(0)}(x) = 1$, $B^{(0)}(x) = 1$, $\gamma^{(0)} = 1$, $L = 0$
- For $r = 0, 1, 2, \dots, \rho - 1$, do:
 - Compute $\Lambda^{(r+1)}(x) = (1 - \alpha_r x)\Lambda^{(r)}(x)$
 - Set $B^{(r+1)}(x) \leftarrow \Lambda^{(r+1)}(x)$, $\gamma^{(r+1)} \leftarrow \gamma^{(r)}$, and $L \leftarrow L + 1$

- For $r = \rho, \rho + 1, \rho + 2, \dots, 2t - 1$, do:
 - Compute $\Delta^{(r)} = \sum_{i=0}^L \Lambda_i^{(r)} \cdot S_{r-i}$
 - Compute $\Lambda^{(r+1)}(x) = \gamma^{(r)} \cdot \Lambda^{(r)}(x) - \Delta^{(r)} \cdot xB^{(r)}(x)$
 - If $\Delta^{(r)} \neq 0$ & $2L \leq r + \rho$, then
 - * Set $B^{(r+1)}(x) \leftarrow \Lambda^{(r)}(x)$ and $L \leftarrow r + \rho + 1 - L$
 - * Set $\gamma^{(r+1)} \leftarrow \Delta^{(r)}$
 - Else
 - * Set $B^{(r+1)}(x) \leftarrow xB^{(r)}(x)$
 - * Set $\gamma^{(r+1)} \leftarrow \gamma^{(r)}$
 - Compute $\Omega^{(2t)}(x) = \Lambda^{(2t)}(x)S(x) \pmod{x^{2t}}$
 - Outputs: $\Lambda^{(2t)}(x)$, $\Omega^{(2t)}(x)$, L
-

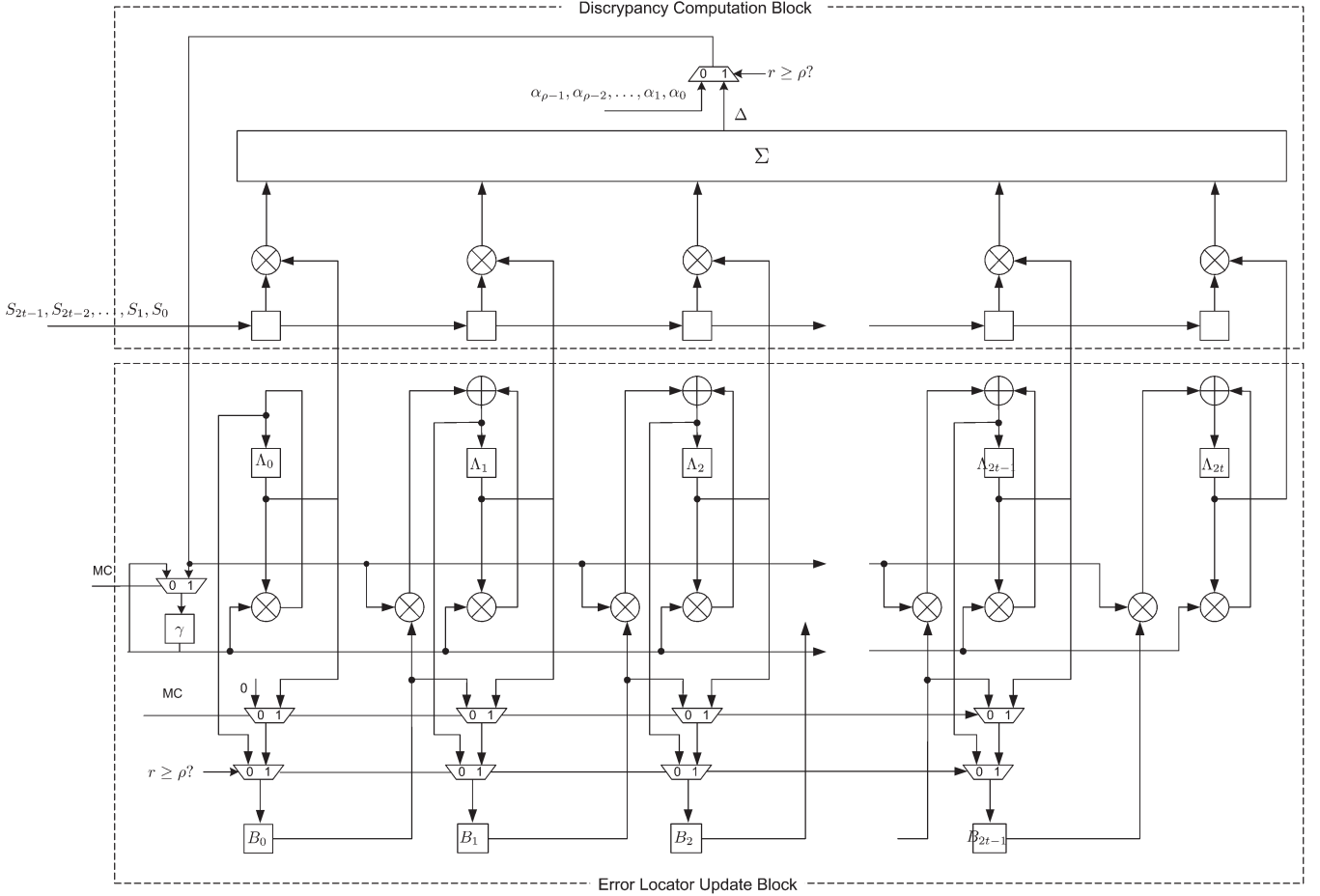
In the above description, superscript “ (r) ” stands for the r -th iteration and subscript “ i ” the i -th coefficient. Note that the Horiguchi-Koetter formula [15, p. 225] is not applicable for error-erasure magnitude evaluation, therefore, it is necessary to compute the error-erasure evaluator polynomial $\Omega(x)$ and apply the Forney formula to evaluate error magnitudes.

In the worst-case of $2t$ erasures, the length/degree of $\Lambda(x)$ is $2t$ and $\Omega(x)$ has a degree of $2t - 1$. Recall for error-only decoding, the length of $\Lambda(x)$ is at most t and the length of $B(x)$ is at most $t - 1$. As a consequence, the corresponding complexity of the Chien search and error evaluation is almost twice as much as that of error-only decoding.

By a first look, the erasure locator polynomial update appears to be totally inhomogeneous to the error-erasure locator polynomial. In [22], IBA is transformed to have a unified iteration of error-erasure locator polynomial.

Reformulated Inversionless Blahut Algorithm (rIBA)

- Inputs: $\mathbf{S} = [S_0, S_1, S_2, \dots, S_{2t-1}]$, $[\alpha_0, \alpha_1, \dots, \alpha_{\rho-1}]$
 - Initialization: $\Lambda^{(0)}(x) = 1$, $B^{(0)}(x) = 1$, $\gamma^{(0)} = 1$, $L_B^{(0)} = L_\Lambda^{(0)} = 0$
 - For $r = 0, 1, 2, \dots, 2t - 1$, do:
 - If $r < \rho$, then set $\Delta^{(r)} \leftarrow \alpha_r$, else compute $\Delta^{(r)} = \sum_{i=0}^{L_\Lambda^{(r)}} \Lambda_i^{(r)} \cdot S_{r-i}$
 - Compute $\Lambda^{(r+1)}(x) = \gamma^{(r)} \cdot \Lambda^{(r)}(x) - \Delta^{(r)} \cdot xB^{(r)}(x)$ (unified)
 - If $r < \rho$
 - * Set $B^{(r+1)}(x) \leftarrow \Lambda^{(r+1)}(x)$
 - * Set $\gamma^{(r+1)} \leftarrow \gamma^{(r)}$
 - * Set $L_B^{(r+1)} \leftarrow r + 1$, $L_\Lambda^{(r+1)} \leftarrow r + 1$
 - Else if $\Delta^{(r)} \neq 0$ & $L_\Lambda^{(r)} \leq L_B^{(r)}$
 - * Set $B^{(r+1)}(x) \leftarrow \Lambda^{(r)}(x)$
 - * Set $\gamma^{(r+1)} \leftarrow \Delta^{(r)}$
 - * Set $L_\Lambda^{(r+1)} \leftarrow L_B^{(r)} + 1$, $L_B^{(r+1)} \leftarrow L_\Lambda^{(r)}$
 - Else
 - * Set $B^{(r+1)}(x) \leftarrow xB^{(r)}(x)$
 - * Set $\gamma^{(r+1)} \leftarrow \gamma^{(r)}$
 - * Set $L_B^{(r+1)} \leftarrow L_B^{(r)} + 1$, $L_\Lambda^{(r+1)} \leftarrow L_\Lambda^{(r)}$
 - Compute $\Omega^{(2t)}(x) = \Lambda^{(2t)}(x)S(x) \pmod{x^{2t}}$
 - Outputs: $\Lambda^{(2t)}(x)$, $\Omega^{(2t)}(x)$, $L_\Lambda^{(2t)}$
-

Fig. 2. The **rIBA** architecture.

The derivative **rIBA** architecture is depicted in Fig. 2. As shown in the figure, it is naturally partitioned into two blocks, discrepancy computation block which computes discrepancies $\Delta^{(r)}$, $r = 1, 2, \dots, 2t$, and error locator update block which iteratively updates the error locator polynomial $\Lambda^{(r)}(x)$, $r = 1, 2, \dots, 2t$. Its detailed measurements of complexity and critical path delay are listed in Table II.

III. ENHANCED INVERSIONLESS B-M ALGORITHM USING A DYNAMIC STOPPING RULE

A. New Insights Into Inversionless B-M Algorithm

IBMA was introduced in Section II-B. In this subsection, we give new insights into IBMA, which will be extensively exploited in the subsequent presentation.

Lemma 1: Let $B^{(r)}(x)$ and $\Lambda^{(r)}(x)$ be obtained during the (inversionless) B-M iterations.

(i). Their lengths satisfy

$$L_B^{(r)} + L_\Lambda^{(r)} = r, \quad 0 \leq r \leq 2t. \quad (6)$$

(ii). Their degrees are both bounded by their lengths, i.e.,

$$\deg(B^{(r)}(x)) \leq L_B^{(r)}, \quad \deg(\Lambda^{(r)}(x)) \leq L_\Lambda^{(r)}. \quad (7)$$

The proof is trivial by induction.

Lemma 2: (i). The length of $B(x)$ may reach as high as $2t-1$.

(ii). Ignoring zeros, the length of $B(x)$ is at most $t-1$.

(iii). If at the r -th ($r < 2t$) iteration $B^{(r)}(x)$ has length $t-1$, then, either more than t errors have occurred (i.e., uncorrectable), or $\Lambda^{(r+1)}(x)$ contains all error locators as inverse of its roots.

Proof: In the worst case scenario there is one symbol error, accordingly the length of $\Lambda(x)$ is 1. Lemma 1 implies that the length of $B(x)$ is $2t-1$.

(ii) When $L_\Lambda > t$, according to Lemma 1, $L_B = 2t - L_\Lambda \leq t-1$; when $L_\Lambda \leq t$, the B-M iteration indicates that $B(x) = B^{(2t)}(x)$ is right-shifted polynomial of a previous $\Lambda^{(r)}(x)$ ($r < 2t$) that has shorter length than $\Lambda(x) = \Lambda^{(2t)}(x)$, therefore, after removing zeros, the length of $B(x)$ is bounded by $L_\Lambda^{(r)} < L_\Lambda \leq t$.

(iii). If at most t errors have occurred, then all inverse error locators must be the all roots of $\Lambda^{(2t)}(x)$, which implies that the length of $\Lambda^{(2t)}(x)$ is at most t . If $r = 2t-1$, then the conclusion trivially holds. We next consider the case $r < 2t-1$. Let $r' > r$ be the smallest number such that $\Delta^{(r')} \neq 0$, then we have

$$\begin{aligned} r' - 2L_\Lambda^{(r)} &= r' - 2r + 2(r - L_\Lambda^{(r)}) = r' - 2r + 2L_B^{(r)} \\ &= r' - 2r + 2(t-1) > 0. \end{aligned}$$

We note that $L_\Lambda^{(r'+1)}$ is updated by

$$\begin{aligned} L_\Lambda^{(r'+1)} &\leftarrow r' + 1 - L_\Lambda^{(r')} = r' + 1 - r + (r - L_\Lambda^{(r')}) \\ &\geq r' + 1 - r + (t-1) > t. \end{aligned}$$

This conflicts with the conclusion that $L_{\Lambda}^{(r'+1)} \leq L_{\Lambda}^{(2t)} \leq t$. Thus, all subsequent discrepancies $\Delta^{(r+1)}, \Delta^{(r+2)}, \dots, \Delta^{(2t-1)}$ must be all zeros, and therefore $\Lambda^{(r+1)}(x)$ differs from $\Lambda^{(2t)}(x)$ by a scalar factor. $\square\square$

Horiguchi–Koetter formula [15, p. 225] directly utilizes the normalized $\Lambda^*(x)$ and $B^*(x)$ generated by the B-M algorithm to evaluate error magnitudes,

$$Y_i = \frac{X_i^{-(2t-1)}}{B^*(X_i^{-1})\Lambda^*(X_i^{-1})}, \quad i = 1, 2, \dots, e. \quad (8)$$

The following lemma generalizes the Horiguchi–Koetter formula to IBMA.

Lemma 3: (i). Given the inversionless B-M iteration in the form

$$\begin{bmatrix} \Lambda^{(r+1)}(x) \\ B^{(r+1)}(x) \end{bmatrix} = \begin{bmatrix} \gamma^{(r)} & -\Delta^{(r)}x \\ \delta & (1-\delta)x \end{bmatrix} \begin{bmatrix} \Lambda^{(r)}(x) \\ B^{(r)}(x) \end{bmatrix} \quad (9)$$

with the initial conditions

$$\Lambda^{(0)}(x) = 1, \quad B^{(0)}(x) = 1, \quad \gamma^{(0)} = 1$$

where $\delta \in \{0, 1\}$ is dependent of each iteration and

$$\gamma^{(r+1)} = \gamma^{(r)}(1-\delta) + \Delta^{(r)}\delta, \quad (10)$$

then

$$\Omega^{(r)}(x)B^{(r)}(x) - \Lambda^{(r)}(x)\theta^{(r)}(x) = \gamma^{(r)}\Lambda_0^{(r)}x^r \quad (11)$$

where

$$\Omega^{(r)}(x) \triangleq \Lambda^{(r)}(x)S(x) \bmod x^r,$$

$$\theta^{(r)}(x) \triangleq B^{(r)}(x)xS(x) - x^r \bmod x^{r+1}$$

(by definition $\Omega^{(0)}(x) = 0$ and $\theta^{(0)}(x) = -1$).

(ii). The error magnitude of a given error locator X_i is determined by

$$Y_i = \frac{\gamma\Lambda_0X_i^{-(2t-1)}}{B(X_i^{-1})\Lambda'(X_i^{-1})}, \quad i = 1, 2, \dots, e. \quad (12)$$

Proof: (i) It is straightforward to show by induction that

$$\begin{bmatrix} \Omega^{(r+1)}(x) \\ \theta^{(r+1)}(x) \end{bmatrix} = \begin{bmatrix} \gamma^{(r)} & -\Delta^{(r)}x \\ \delta & (1-\delta)x \end{bmatrix} \begin{bmatrix} \Omega^{(r)}(x) \\ \theta^{(r)}(x) \end{bmatrix}.$$

We thus obtain the following single iteration rule

$$\begin{bmatrix} \Omega^{(r+1)}(x) & \Lambda^{(r+1)}(x) \\ \theta^{(r+1)}(x) & B^{(r+1)}(x) \end{bmatrix} = \begin{bmatrix} \gamma^{(K)} & -\Delta^{(K)}x \\ \delta & (1-\delta)x \end{bmatrix} \cdot \begin{bmatrix} \Omega^{(r)}(x) & \Lambda^{(r)}(x) \\ \theta^{(r)}(x) & B^{(r)}(x) \end{bmatrix}.$$

We are now ready to prove (11) by induction. When $r = 1$,

$$\begin{aligned} & \Omega^{(1)}(x)B^{(1)}(x) - \Lambda^{(1)}(x)\theta^{(1)}(x) \\ &= \begin{vmatrix} \Omega^{(1)}(x) & \Lambda^{(1)}(x) \\ \theta^{(1)}(x) & B^{(1)}(x) \end{vmatrix} \\ &= \begin{vmatrix} \gamma^{(0)} & -\Delta^{(0)}x \\ \delta & (1-\delta)x \end{vmatrix} \cdot \begin{vmatrix} \Omega^{(0)}(x) & \Lambda^{(0)}(x) \\ \theta^{(0)}(x) & B^{(0)}(x) \end{vmatrix} \\ &= [\gamma^{(0)}(1-\delta) + \Delta^{(0)}\delta] \cdot x \\ &= \gamma^{(1)}\Lambda_0^{(1)}x, \end{aligned}$$

where the last equality is due to (10) and $\Lambda_0^{(1)} = 1$. Suppose that it is true for $r = K$, then, we have for $r = K + 1$,

$$\begin{aligned} & \Omega^{(K+1)}(x)B^{(K+1)}(x) - \Lambda^{(K+1)}(x)\theta^{(K+1)}(x) \\ &= \begin{vmatrix} \Omega^{(K+1)}(x) & \Lambda^{(K+1)}(x) \\ \theta^{(K+1)}(x) & B^{(K+1)}(x) \end{vmatrix} \\ &= \begin{vmatrix} \gamma^{(K)} & -\Delta^{(K)}x \\ \delta & (1-\delta)x \end{vmatrix} \cdot \gamma^{(K)}\Lambda_0^{(K)}x^K \\ &= \gamma^{(K+1)}\Lambda_0^{(K+1)}x^K, \end{aligned}$$

where the last equality is due to (10) and $\Lambda_0^{(K+1)} = \Lambda_0^{(K)}\gamma^{(K)}$. We therefore conclude Part (i).

Part (ii) is a straightforward combination of the Forney formula (5) and the result of Part (i) (11), in conjunction with the fact that $\Lambda(X^{-1}) = 0$. $\square\square$

Given operating in binary extension field $\text{GF}(2^m)$, the computation of error magnitudes can be simplified further to be

$$Y_i = \frac{\gamma\Lambda_0X_i^{-(2t)}}{B(X_i^{-1})\Lambda_{\text{odd}}(X_i^{-1})}, \quad i = 1, 2, \dots, e. \quad (13)$$

An intuitive interpretation of the above generalization of the Horiguchi–Koetter formula is to view $\gamma^{-1}B(x)$ as the normalized auxiliary polynomial $B^*(x)$ and $\Lambda_0^{-1}\Lambda(x)$ as the normalized error locator polynomial $\Lambda^*(x)$, then the above generalized formula is reduced to the original formula (8).

B. Enhanced Inversionless B-M Algorithm

When the number of errors e is less than t , the length of $\Lambda(x)$ satisfies $L_{\Lambda} = e$. Then, Lemma 1 implies that the length of $B(x)$ is $L_B = 2t - e > t$. We show in the following lemma that in such cases it is possible to terminate the B-M iteration earlier while limiting the length of $B(x)$ to $t - 1$, which is the minimum value without causing performance degradation.

Lemma 4: Given the number of errors $e \leq t$.

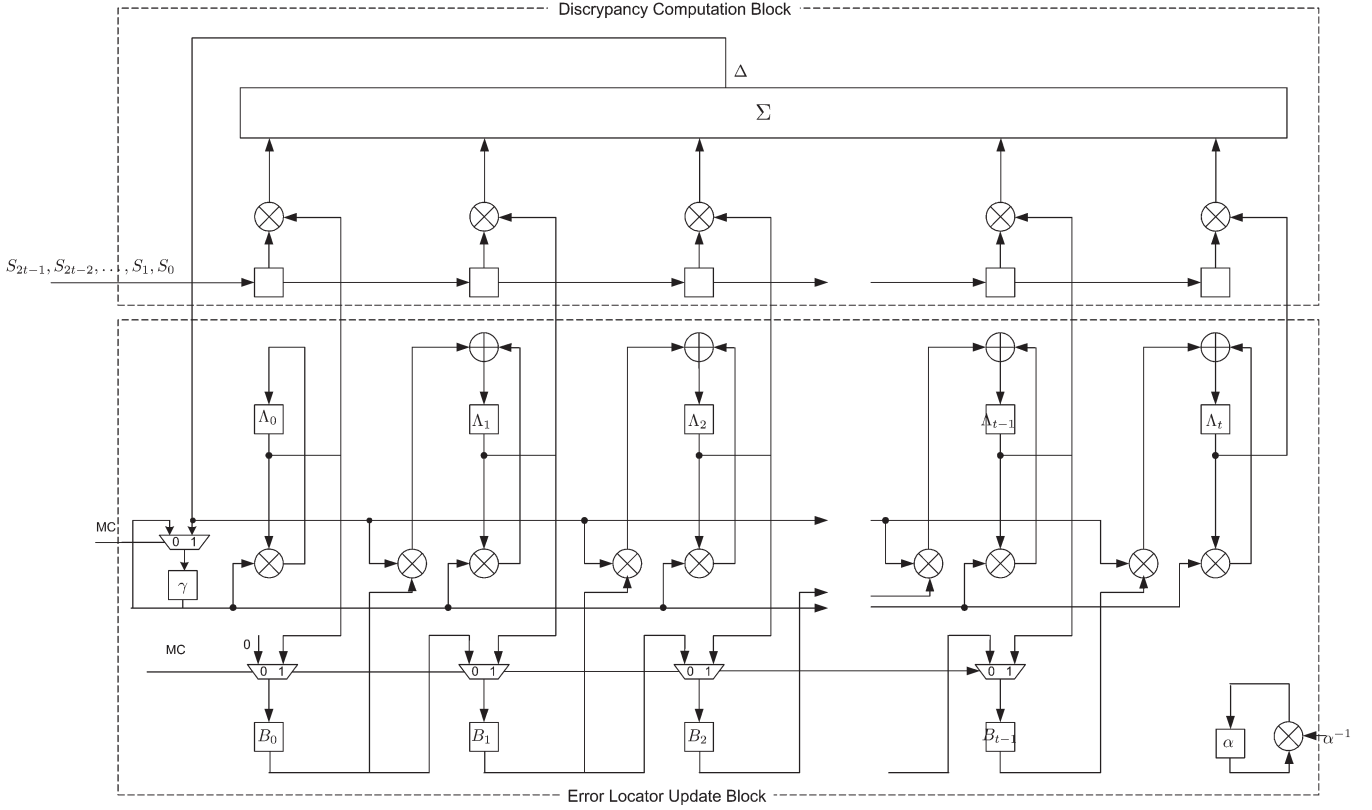
(i). $B^{(t+e-1)}(x)$ has length $t - 1$, and the error locators X_i^{-1} , $i = 1, 2, \dots, e$, are all roots of $\Lambda^{(t+e)}(x)$.

(ii). The error magnitudes are determined by

$$Y_i = \frac{\gamma^{(t+e-1)}\Lambda_0^{(t+e)} \cdot X_i^{-t-e+2}}{B^{(t+e-1)}(X_i^{-1}) \cdot [\Lambda^{(t+e)}]'(X_i^{-1})}, \quad i = 1, 2, \dots, e. \quad (14)$$

Proof: The conclusion trivially holds for $e = t$. We next show for the case $e < t$.

(i). After $2e$ iterations, the length of $\Lambda^{(2e)}(x)$ is e and $\Lambda^{(2e)}(x)$ contains all error locators as inverse of its roots. Lemma 1

Fig. 3. The **eIBMA** architecture.

indicates that the length of $B^{(2e)}(x)$ is e as well. From $(2e+1)$ -th iteration to $(2t)$ -th, the discrepancy values are always zero and $\Lambda^{(r)}(x)$, $r = 2e+1, 2e+2, \dots, 2t$ are unchanged ignoring a scalar factor. Therefore, at the $t+e-1$ -th iteration, the length of $B^{(t+e-1)}(x)$ is $t+e-1 - \deg(\Lambda^{(t+e-1)}) = t-1$. On the other hand, $\Lambda^{(t+e)}(x)$ differs from $\Lambda^{(2e)}(x)$ by a scalar factor and thus contains all error locators as inverse of its roots. The proof of (ii) follows from Lemma 3.(i). $\square\square$

In view of the above Lemmas 2.(iii) and 4, we propose the following enhancements toward IBMA.

Enhanced Inversionless B-M Algorithm (eIBMA)

- Inputs: $\mathbf{S} = [S_0, S_1, S_2, \dots, S_{2t-1}]$
- Initialization: $\Lambda^{(0)}(x) = 1$, $B^{(0)}(x) = 1$, $\gamma^{(0)} = 1$, $z = \alpha$, $L = 0$
- For $r = 0, 1, 2, \dots, 2t-1$, do:
 - Set $z \leftarrow z \cdot \alpha^{-1}$
 - Compute $\Delta^{(r)} = \sum_{i=0}^L \Lambda_i^{(r)} \cdot S_{r-i}$
 - Compute $\Lambda^{(r+1)}(x) = \gamma^{(r)} \cdot \Lambda^{(r)}(x) - \Delta^{(r)} \cdot xB^{(r)}(x)$
 - If $\Delta^{(r)} \neq 0$ and $2L \leq r$, then
 - * Set $B^{(r+1)}(x) \leftarrow \Lambda^{(r)}(x)$ and $L \leftarrow r+1-L$
 - * Set $\gamma^{(r+1)} \leftarrow \Delta^{(r)}$
 - Else
 - * If $r-L = t-1$ then terminate (new)
 - * Else set $B^{(r+1)}(x) \leftarrow xB^{(r)}(x)$, $\gamma^{(r+1)} \leftarrow \gamma^{(r)}$
- Outputs: $\mathbf{\Lambda} = [\Lambda_0^{(r+1)}, \Lambda_1^{(r+1)}, \dots, \Lambda_t^{(r+1)}]$, $\mathbf{B} = [B_0^{(r)}, B_1^{(r)}, \dots, B_{t-1}^{(r)}]$, $\gamma = \gamma^{(r)}$, L, z .

Note that the proposed modification also allows to save power dynamically. In particular, when the average number of errors is much less than t , the operation power is reduced by almost half.

The architecture to implement the proposed eIBMA is depicted in Fig. 3 (Throughout the paper, we use an acronym to represent for an algorithm and a bolded acronym for the associated architecture, e.g., eIBMA represents the algorithm whereas **eIBMA** represents its associated architecture). As shown in the figure, it is naturally partitioned into two blocks, discrepancy computation block which computes discrepancies $\Delta^{(r)}$, $r = 1, 2, \dots, 2t$, and error locator update block which iteratively updates the error locator polynomial $\Lambda^{(r)}(x)$, $r = 1, 2, \dots, 2t$. The discrepancy computation block requires $t+1$ multipliers and t adders, $2t$ registers, ignoring miscellaneous other circuitry in the control (counters, arithmetic adder or ring counter, OR gates, latches). Its critical path has 1 multiplier and $\lceil \log_2(t+1) \rceil$ adders assuming that t additions is computed via a binary tree structure.

The error locator update block requires $2t+1$ multipliers (note that the update of Λ_0 requires only 1 multiplier while the update of each of the remaining t terms $\Lambda_1, \Lambda_2, \dots, \Lambda_t$, requires 2 multipliers), $t+1$ adders, $2t+1$ latches (note $t+1$ latches for $\Lambda(x)$ and t latches for $B(x)$), and t multiplexers. Clearly, the critical path of the error locator update block contains one multiplier and one adder. The separate loop logic of z will be used in error evaluation. Note the control signal **MC** stands for the Boolean operation " $\Delta^{(r)} \neq 0$ and $2r \leq L$ ".

In total, the **eIBMA** architecture contains $3t+2$ multipliers, $2t+1$ adders, $2t+1$ latches, and t multiplexers. As can be

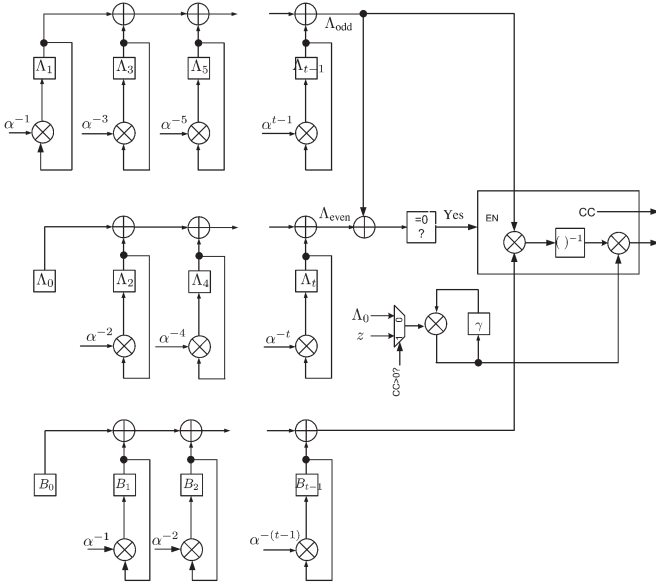


Fig. 4. The eCSEE architecture.

inspected from the figure, the critical path delay of **eIBMA** is given by

$$T_{\text{crit-path}} = 2T_{\text{mult}} + (1 + \lceil \log_2(t+1) \rceil) \cdot T_{\text{add}}. \quad (15)$$

The following pseudo code of the enhanced Chien search and error evaluation (eCSEE) takes into account the proposed simplifications.

Enhanced Chien Search & Error Evaluation (eCSEE)

- Inputs: $\mathbf{\Lambda} = [\Lambda_0, \Lambda_1, \dots, \Lambda_t]$, $\mathbf{B} = [B_0, B_1, \dots, B_{t-1}]$, γ, L, z
 - Initialization: $Z = \gamma$, $\lambda_i = \Lambda_i$, $i = 0, 1, 2, \dots, t$, $b_i = B_i$, $i = 0, 1, 2, \dots, t-1$, $e = 0$
 - For $j = 0, 1, 2, \dots, n-1$, do:
 - If $j = 0$ then set $Z \leftarrow Z \cdot \Lambda_0$, else set $Z \leftarrow Z \cdot z$
 - Compute $\Lambda_{\text{odd}} = \lambda_1 + \lambda_3 + \lambda_5 + \dots + \lambda_{t+\delta_t-1}$ /*
 $\Lambda_{\text{odd}} = \Lambda_{\text{odd}}(\alpha^{-j})$ */
 - Set $\lambda_i \leftarrow \lambda_i \cdot \alpha^{-i}$, $i = 1, 3, 5, \dots, t + \delta_t - 1$
 - Compute $\Lambda_{\text{even}} = \lambda_0 + \lambda_2 + \lambda_4 + \dots + \lambda_{t-\delta_t}$ /*
 $\Lambda_{\text{even}} = \Lambda_{\text{even}}(\alpha^{-j})$ */
 - Set $\lambda_i \leftarrow \lambda_i \cdot \alpha^{-i}$, $i = 2, 4, \dots, t - \delta_t$
 - Compute $B = b_0 + b_1 + b_2 + \dots + b_{t-1}$ /* $B = B(\alpha^{-j})$ */
 - Set $b_i \leftarrow b_i \cdot \alpha^{-i}$, $i = 1, 2, \dots, t-1$
 - If $\Lambda_{\text{odd}} = \Lambda_{\text{even}}$, then
 - * Compute the error magnitude $Y = (B\Lambda_{\text{odd}})^{-1}Z$
 - * Store the error pair (j, Y)
 - * Set $e \leftarrow e + 1$
 - Outputs: If $e < L$ then claim failure, otherwise output $(j_1, Y_1), (j_2, Y_2), \dots, (j_e, Y_e)$
-

Note that the update of λ_i , $i = 1, 2, \dots, t$ and b_i , $i = 1, 2, \dots, t-1$, involve only fixed value multiplication, the so-called half-multiplier, whose complexity is much smaller than (roughly speaking, one third of) regular multiplier. The **eCSEE**

architecture is depicted in Fig. 4, where t is assumed to be an even number. We denote “CC” by the clock cycles, which indicates the index of a root (i.e., an error location). Note that the evaluation of $\Lambda'(x)$, $\Lambda(x)$, $B(x)$ uses half-multipliers (i.e., one input end of the multiplier is fixed) whose complexity is roughly one third of a full-multiplier. By setting γ to the initial value of the loop logic and multiplying Λ_0 in the first iteration (which is otherwise a null operation), the computation of $\gamma \Lambda_0 z^i$ is effectively achieved with one full-multiplier and one multiplexer, as would require three full-multipliers in a straightforward implementation. In total, the **eCSEE** architecture uses $2t - 1$ half-multipliers, 3 full-multipliers, 1 inverter, $2t - 2$ adders, and $2t + 3$ latches/registers. Note that the error evaluator polynomial $\Omega(x)$ has the same degree bound as $B(x)$. Indeed, the above hardware complexity is comparable to the alternative **CSEE** that uses Forney formula given in (5). Mainly, the latter employs one less full-multiplier.

IV. PARALLEL INVERSIONLESS B-M ALGORITHMS AND THEIR SYSTOLIC ARCHITECTURES

In view of the **eIBMA**, it suffers from low clock frequency due to the cascade of discrepancy computation block and error locator update block, resulting in a long critical path delay of $2T_{\text{mult}} + \lceil \log_2(2t+2) \rceil T_{\text{add}}$. This is particularly undesirable in a configurable implementation, where the critical path is dictated by the largest t . In this section we investigate the variations of IBMA and their associated architectures in which the two blocks, discrepancy computation and polynomial update, are implemented in parallel so that low latency is achieved. The pioneer work has been established in [11].

A. Parallel Inversionless B-M Algorithms

In [11], the authors proposed parallel B-M algorithm in which the discrepancy computation and polynomial updates are operated simultaneously. Note that in the IBMA, the discrepancy value $\Delta^{(r)}$ is computed based on the error locator polynomial $\Lambda^{(r)}(x)$, as is the essential cause of the high latency of the algorithm. We next review the algorithm to generate discrepancies iteratively by [11].

Define a left-shift operator “ $\mathcal{L}$ ” of a polynomial such that

$$[\mathcal{L}_r A](x) \triangleq [A(x) - (A(x) \bmod x^r)] / x^r \quad (16)$$

An alternative interpretation gives more insight. Let $\mathbf{A} = [A_0, A_1, A_2, \dots, A_t]$ be the vector representation of the polynomial $A(x)$, then $\mathcal{L}_r \mathbf{A} = [A_r, A_{r+1}, \dots, A_t]$.

The following block of pseudo code iteratively generates the discrepancy sequence [11].

Iterative Discrepancy Generation Algorithm

- Input: $\mathbf{S} = [S_0, S_1, S_2, \dots, S_{2t-1}]$
- Initialization: $\hat{\Omega}^{(0)}(x) = S(x)$, $\hat{\Theta}^{(0)}(x) = S(x)$, $\gamma^{(0)} = 1$, $L = 0$
- For $r = 0, 1, 2, \dots, 2t-1$, do:
 - $\hat{\Omega}^{(r+1)}(x) = \gamma^{(r)} \cdot [\mathcal{L}_1 \hat{\Omega}^{(r)}](x) - \hat{\Omega}_0^{(r)} \cdot \hat{\Theta}^{(r)}(x)$

- If $\hat{\Omega}_0^{(r)} \neq 0$ and $2L \leq r$, then
 - * Set $\hat{\Theta}^{(r+1)}(x) \leftarrow [\mathcal{L}_1 \hat{\Omega}^{(r)}](x)$
 - * Set $L \leftarrow r + 1 - L$
 - * Set $\gamma^{(r+1)} \leftarrow \hat{\Omega}_0^{(r)}$
- Else
 - * Set $\hat{\Theta}^{(r+1)}(x) \leftarrow \hat{\Theta}^{(r)}(x)$
 - * Set $\gamma^{(r+1)} \leftarrow \gamma^{(r)}$
- Output: $\Delta = [\hat{\Omega}_0^{(0)}, \hat{\Omega}_0^{(1)}, \dots, \hat{\Omega}_0^{(2t-1)}]$

Note that $\hat{\Omega}^{(r)}(x)$ and $\hat{\Theta}^{(r)}(x)$ is a left-shifted polynomial of $\Lambda^{(r)}(x)S(x)$ and $B^{(r)}(x)S(x)$ respectively, more specifically,

$$\begin{aligned}\hat{\Omega}^{(r)}(x) &= [\mathcal{L}_r(\Lambda^{(r)}S)](x), \\ \hat{\Theta}^{(r)}(x) &= [\mathcal{L}_r(B^{(r)}S)](x),\end{aligned}$$

where $\Lambda^{(r)}(x)$ and $B^{(r)}(x)$ denote the error locator polynomial and its auxiliary polynomial generated during the r -th iteration of IBMA [11].

We propose the following modifications to the above algorithm and characterize consequent properties thereafter.

New Iterative Discrepancy Generation Algorithm

- Input: $\mathbf{S} = [S_0, S_1, S_2, \dots, S_{2t-1}]$
- Initialization: $\hat{\Theta}^{(0)}(x) = S_0 + S_1x + \dots + S_{2t-2}x^{2t-2}$ (modified), $\hat{\Omega}^{(0)}(x) = S_0 + S_1x + \dots + S_{2t-2}x^{2t-2} + S_{2t-1}x^{2t-1}$, $\gamma^{(0)} = 1$, $L = 0$
- For $r = 0, 1, 2, \dots, 2t - 1$, do:
 - $\hat{\Omega}^{(r+1)}(x) = \gamma^{(r)} \cdot [\mathcal{L}_1 \hat{\Omega}^{(r)}](x) - \hat{\Omega}_0^{(r)} \cdot \hat{\Theta}^{(r)}(x)$
 - If $\hat{\Omega}_0^{(r)} \neq 0$ and $2L \leq r$, then
 - * Set $\hat{\Theta}^{(r+1)}(x) \leftarrow [\mathcal{L}_1 \hat{\Omega}^{(r)}](x)$
 - * Set $L \leftarrow r + 1 - L$
 - * Set $\gamma^{(r+1)} \leftarrow \hat{\Omega}_0^{(r)}$
 - Else
 - * Set $\hat{\Theta}^{(r+1)}(x) \leftarrow \hat{\Theta}^{(r)}(x)$
 - * Set $\gamma^{(r+1)} \leftarrow \gamma^{(r)}$
 - Set $\hat{\Theta}_{2t-r-2}^{(r+1)} = 0$ (new)
- Output: $\Delta = [\hat{\Omega}_0^{(0)}, \hat{\Omega}_0^{(1)}, \dots, \hat{\Omega}_0^{(2t-1)}]$

Lemma 5: (i). $\{\hat{\Omega}_i^{(r)}\}_{i=0}^{2t-1-r}$ and $\{\hat{\Theta}_i^{(r)}\}_{i=0}^{2t-2-r}$, $r = 0, 1, 2, \dots, 2t$, are intact to the proposed modifications.

(ii). The proposed modifications do not change the discrepancy output.

Proof: We show Part (i) by induction. It is certainly true for $r = 0$. When $r = 1$, it is readily inspected that $\{\hat{\Omega}_i^{(1)}\}_{i=0}^{2t-2}$, $\{\hat{\Theta}_i^{(1)}\}_{i=0}^{2t-3}$, are intact, so the proposition holds for $r = 1$ as well.

Suppose it holds for $r \leq K$, we consider the case $r = K + 1$. Note that $\hat{\Omega}^{(K+1)}(x)$ is determined by

$$\hat{\Omega}^{(K+1)}(x) = \gamma^{(K)} \cdot [\mathcal{L}_1 \hat{\Omega}^{(K)}](x) - \hat{\Omega}_0^{(K)} \cdot \hat{\Theta}^{(K)}(x).$$

Since $\gamma^{(K)}$ is a function of $\hat{\Omega}_0^{(0)}, \hat{\Omega}_0^{(1)}, \dots, \hat{\Omega}_0^{(K-1)}$, it is intact. Based on the assumption, $\{\hat{\Omega}_i^{(K+1)}\}_{i=0}^{2t-2-K}$ are intact. On the other hand, $\hat{\Theta}^{(K+1)}(x)$ is either $[\mathcal{L}_1 \hat{\Omega}^{(K)}](x)$ or $\hat{\Theta}^{(K)}(x)$, thus,

again by the assumption, $\{\hat{\Theta}_i^{(K+1)}\}_{i=0}^{2t-3-K}$ are intact. Since $\hat{\Theta}_{2t-K-2}^{(K+1)}$ is not included in the set $\{\hat{\Theta}_i^{(K+1)}\}_{i=0}^{2t-3-K}$, none of its elements is reset to 0 during the $(K + 1)$ -th iteration. That is, the proposition also holds for the case $r = K + 1$. We thus conclude Part (i).

Part (ii) is clearly a corollary of Part (i). $\square\square$

Lemma 6: (i). The proposed modifications limit the nonzero coefficients of $\hat{\Omega}(x)$ and $\hat{\Theta}(x)$ along with $2t$ iterations to the shape of upper triangular. More specifically,

$$\hat{\Omega}_{2t-r-1}^{(r)} = \hat{\Omega}_{2t-r}^{(r)} = \dots = \hat{\Omega}_{2t-1}^{(r)} = 0, \quad (17)$$

$$\hat{\Theta}_{2t-r-2}^{(r)} W = \hat{\Theta}_{2t-r-1}^{(r)} = \dots = \hat{\Theta}_{2t-1}^{(r)} = 0 \quad (18)$$

for $r = 1, 2, \dots, 2t$.

(ii). When the number of errors $e < t$,

$$\hat{\Omega}^{(r)}(x) = 0, \quad (19)$$

for $r = 2e, 2e + 1, \dots, 2t - 1$.

Proof: Part (i) can be shown by induction straightforwardly.

(ii). Clearly, $\Delta^{(2e)}(x)$ contains all error locators as inverse of its roots. Consequently, we have

$$\begin{aligned}[\Delta^{(2e)}, \Delta^{(2e+1)}, \dots, \Delta^{(2t-1)}] \\ = [\hat{\Omega}_0^{(2e)}, \hat{\Omega}_0^{(2e+1)}, \dots, \hat{\Omega}_0^{(2t-1)}] = \mathbf{0}.\end{aligned}$$

$\hat{\Omega}_0^{(2e)} = 0$ indicates that $\hat{\Omega}_0^{(2e+1)} = \gamma^{(2e)} \hat{\Omega}_1^{(2e)}$, thus, $\hat{\Omega}_1^{(2e)} = 0$, and so forth, we obtain consequently $\hat{\Omega}_2^{(2e)} = 0$, $\hat{\Omega}_3^{(2e)} = 0$, $\dots$, $\hat{\Omega}_{2t-2e-2}^{(2e)} = 0$. In conjunction with Part (i), we have $\hat{\Omega}^{(2e)}(x) = 0$. Subsequently we obtain $\hat{\Omega}^{(2e+1)}(x) = 0$, due to $\hat{\Omega}^{(2e)}(x) = 0$ and $\hat{\Omega}_0^{(2e)} = 0$. By induction, it holds that $\hat{\Omega}^{(r)}(x) = 0$, $r = 2e + 2, 2e + 3, \dots, 2t - 1$. We thus conclude (ii). $\square\square$

In order to be homogeneous to the discrepancy generation, error locator polynomial update is also arranged in a left shifted fashion.

Left-Shifted Polynomial Update With Genie Discrepancy Sequence

- Input: $\Delta = [\Delta^{(0)}, \Delta^{(1)}, \Delta^{(2)}, \dots, \Delta^{(2t-1)}]$
- Initialization: $\Lambda^{(0)}(x) = B^{(0)}(x) = x^{2t}$, $\gamma^{(0)} = 1$, $L = 0$
- For $r = 0, 1, 2, \dots, 2t - 1$, do:
 - $\Lambda^{(r+1)}(x) = \gamma^{(r)} \cdot [\mathcal{L}_1 \Lambda^{(r)}](x) - \Delta^{(r)} \cdot B^{(r)}(x)$
 - If $\Delta^{(r)} \neq 0$ and $2L \leq r$, then
 - * Set $B^{(r+1)}(x) \leftarrow [\mathcal{L}_1 \Lambda^{(r)}](x)$
 - * Set $L \leftarrow r + 1 - L$
 - * Set $\gamma^{(r+1)} \leftarrow \Delta^{(r)}$
 - Else
 - * Set $B^{(r+1)}(x) \leftarrow B^{(r)}(x)$
 - * Set $\gamma^{(r+1)} \leftarrow \gamma^{(r)}$
- Outputs: $\mathbf{A} = [\Lambda_0^{(2t)}, \Lambda_1^{(2t)}, \Lambda_2^{(2t)}, \dots, \Lambda_t^{(2t)}]$, $\mathbf{B} = [B_0^{(2t)}, B_1^{(2t)}, \dots, B_{2t-1}^{(2t)}]$, $\gamma = \gamma^{(2t)}$, L

It is now handy to merge the above two component algorithms into a single homogeneous algorithm as follows. In addition, condition checking in the control unit can be significantly simplified by explicitly tracking the length of $B(x)$.

Parallel Inversionless B-M Algorithm (PIBMA)

- Input: $\mathbf{S} = [S_0, S_1, S_2, \dots, S_{2t-1}]$
- Initialization: $\hat{\Theta}^{(0)}(x) = S_0 + S_1x + \dots + S_{2t-2}x^{2t-2} + x^{2t}$, $\hat{\Omega}^{(0)}(x) = S_0 + S_1x + \dots + S_{2t-2}x^{2t-2} + S_{2t-1}x^{2t-1} + x^{2t}$, $\gamma^{(0)} = 1, L_{\Lambda}^{(0)} = L_B^{(0)} = 0$
- For $r = 0, 1, 2, \dots, 2t - 1$, do:
 - $\hat{\Omega}^{(r+1)}(x) = \gamma^{(r)} \cdot [\mathcal{L}_1 \hat{\Omega}^{(r)}](x) - \hat{\Omega}_0^{(r)} \cdot \hat{\Theta}^{(r)}(x)$
 - If $\hat{\Omega}_0^{(r)} \neq 0$ and $L_{\Lambda}^{(r)} \leq L_B^{(r)}$, then
 - * Set $\hat{\Theta}^{(r+1)}(x) \leftarrow [\mathcal{L}_1 \hat{\Omega}^{(r)}](x)$
 - * Set $L_{\Lambda}^{(r+1)} \leftarrow L_B^{(r)} + 1, L_B^{(r+1)} \leftarrow L_{\Lambda}^{(r)}$
 - * Set $\gamma^{(r+1)} \leftarrow \hat{\Omega}_0^{(r)}$
 - Else
 - * Set $\hat{\Theta}^{(r+1)}(x) \leftarrow \hat{\Theta}^{(r)}(x)$
 - * Set $L_{\Lambda}^{(r+1)} \leftarrow L_{\Lambda}^{(r)}, L_B^{(r+1)} \leftarrow L_B^{(r)} + 1$
 - * Set $\gamma^{(r+1)} \leftarrow \gamma^{(r)}$
 - Set $\hat{\Theta}_{2t-r-2}^{(r+1)} \leftarrow 0$
- Outputs: $\mathbf{\Lambda} = [\hat{\Omega}_0^{(2t)}, \hat{\Omega}_1^{(2t)}, \dots, \hat{\Omega}_t^{(2t)}], \mathbf{B} = [\hat{\Theta}_0^{(2t)}, \hat{\Theta}_1^{(2t)}, \dots, \hat{\Theta}_{2t-1}^{(2t)}], \gamma^{(2t)}, L_{\Lambda}^{(2t)}$

In order to limit the degree of $B(x)$ to $t - 1$, we next introduce an auxiliary criterion and again utilize a separate loop logic z to accumulate the unknown $\alpha^{-(t+e-2)}$ which will be used in the error evaluation (refer to Lemma 4). The enhanced algorithm is described below.

Enhanced Parallel Inversionless B-M Algorithm (ePIBMA)

- Input: $\mathbf{S} = [S_0, S_1, S_2, \dots, S_{2t-1}]$
- Initialization: $\hat{\Theta}^{(0)}(x) = S_0 + S_1x + \dots + S_{2t-2}x^{2t-2} + x^{2t}$, $\hat{\Omega}^{(0)}(x) = S_0 + S_1x + \dots + S_{2t-2}x^{2t-2} + S_{2t-1}x^{2t-1} + x^{2t}$, $\gamma^{(0)} = 1, L_{\Lambda}^{(0)} = L_B^{(0)} = 0, z = 1$
- For $r = 0, 1, 2, \dots, 2t - 1$, do:
 - $\hat{\Omega}^{(r+1)}(x) = \gamma^{(r)} \cdot [\mathcal{L}_1 \hat{\Omega}^{(r)}](x) - \hat{\Omega}_0^{(r)} \cdot \hat{\Theta}^{(r)}(x)$
 - If $\hat{\Omega}_0^{(r)} \neq 0$ & $L_{\Lambda}^{(r)} \leq L_B^{(r)}$, then
 - * Set $\hat{\Theta}^{(r+1)}(x) \leftarrow [\mathcal{L}_1 \hat{\Omega}^{(r)}](x)$
 - * Set $L_{\Lambda}^{(r+1)} \leftarrow L_B^{(r)} + 1, L_B^{(r+1)} \leftarrow L_{\Lambda}^{(r)}$
 - * Set $\gamma^{(r+1)} \leftarrow \hat{\Omega}_0^{(r)}, z \leftarrow \alpha^{-1}z$
 - Else
 - * If $L_B^{(r)} = t - 1$, then set $\hat{\Theta}^{(r+1)}(x) \leftarrow [\mathcal{L}_1 \hat{\Theta}^{(r)}](x)$, $L_B^{(r+1)} \leftarrow L_B^{(r)}$ (new)
 - * Else set $\hat{\Theta}^{(r+1)}(x) \leftarrow \hat{\Theta}^{(r)}(x), z \leftarrow z \cdot \alpha^{-1}, L_B^{(r+1)} \leftarrow L_B^{(r)} + 1$
 - * Set $\gamma^{(r+1)} \leftarrow \gamma^{(r)}, L_{\Lambda}^{(r+1)} \leftarrow L_{\Lambda}^{(r)}$
 - Set $\hat{\Theta}_{2t-r-2}^{(r+1)} = 0$
- Outputs: $\mathbf{\Lambda} = [\hat{\Omega}_0^{(2t)}, \hat{\Omega}_1^{(2t)}, \dots, \hat{\Omega}_t^{(2t)}], \mathbf{B} = [\hat{\Theta}_0^{(2t)}, \hat{\Theta}_1^{(2t)}, \dots, \hat{\Theta}_{t-1}^{(2t)}], \gamma^{(2t)}, L_{\Lambda}^{(2t)}, z$

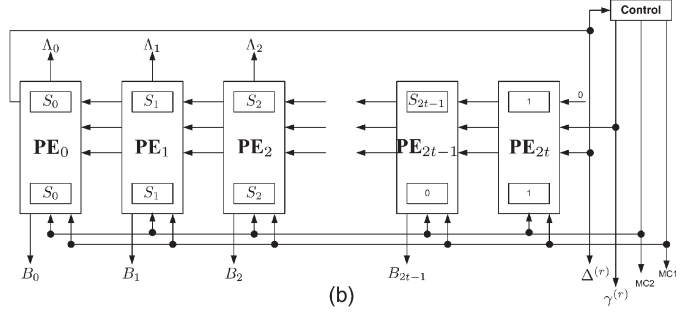
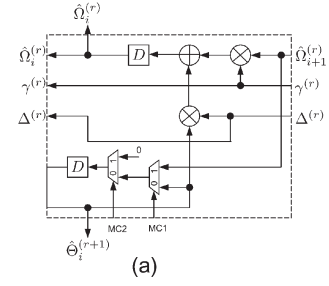


Fig. 5. The systolic **PIBMA** architecture. (a) The **PE** processor. (b) **PIBMA** block diagram.

The properness of PIBMA and ePIBMA is asserted by Lemmas 5 and 6. Through sequentially nullifying one term, $\hat{\Theta}_{2t-r-2}^{(r+1)} \leftarrow 0, 0 \leq r < 2t$, in the array of the discrepancy computation unit, the last operational term is naturally terminated at each iteration and thus the number of operational terms exhibits the shape of reverse triangle along the axis of iteration number, as shown by Lemma 6. This special property enables the array of discrepancy computation and the array of polynomial iteration to “seamlessly” concatenate into an inseparable array. Specifically, the degree of $\hat{\Omega}^{(r)}(x)$ is limited to $2t - r - 1$ (Lemma 6), whereas $\Lambda^{(r)}(x)$ starts from x^{2t-r} due to left shift by one at each iteration, resulting in no gap but also no overlapping between the two concatenated polynomials. It is worth noting that they are power efficient, specifically, when the number of errors $e < t$, after $2e$ iterations, the update logic $\hat{\Omega}^{(r+1)}(x) = \gamma^{(r)} \cdot [\mathcal{L}_1 \hat{\Omega}^{(r)}](x) - \hat{\Omega}_0^{(r)} \cdot \hat{\Theta}^{(r)}(x)$ where $r \geq 2e$, invokes only $e + 1$ multipliers (which are associated with $\Lambda(x)$), while the remaining $2t + 1 - e$ ones are automatically disabled. That is, after $2e$ iterations, the algorithm acts as a shift register array.

B. Systolic Architectures

The **PIBMA** and **ePIBMA** architectures are depicted in Figs. 5 and 6, respectively. Both figures contain an array of $2t + 1$ homogeneous systolic processor elements (**PE**). The control signal **MC1** represents the Boolean operation “ $\Omega^{(r)} \neq 0$ and $L_{\Lambda}^{(r)} \leq L_B^{(r)}$ ”. To shorten the latency of the comparison $L_{\Lambda}^{(r)} \leq L_B^{(r)}$, it is beneficial to introduce a dedicated term $L_{\text{diff}}^{(r)} \triangleq L_{\Lambda}^{(r)} - L_B^{(r)} - 1$. It is initialized to -1 and iteratively updated. Then, the condition $L_{\Lambda}^{(r)} \leq L_B^{(r)}$ is equivalent to $L_{\text{diff}}^{(r)} < 0$, which becomes simply to compare the sign bit of $L_{\text{diff}}^{(r)}$. There are various ways to implement $\hat{\Theta}_{2t-r-2}^{(r+1)} = 0, r = 0, 1, 2, \dots, 2t - 1$. Herein we use a $2t$ -dimension unit vector with the only one initially set

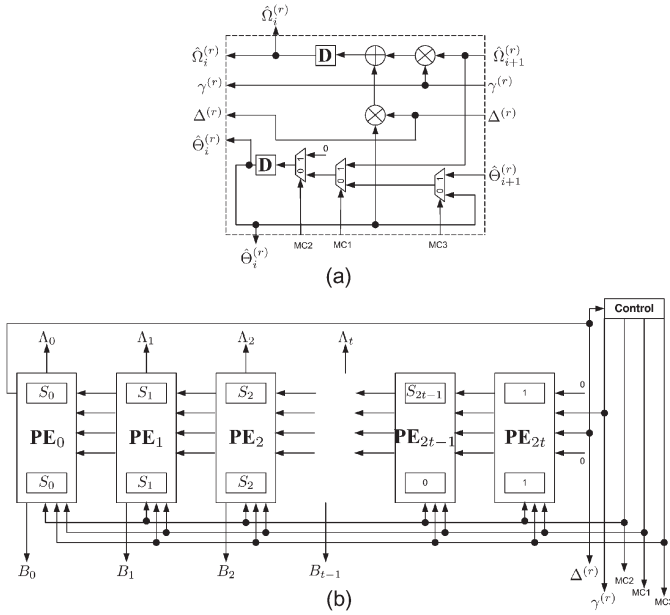


Fig. 6. The systolic **ePIBMA** architecture. (a) The **PE** processor. (b) **ePIBMA** block diagram.

to the right-most position. In each iteration, the element $\hat{\Theta}_i$ is set to 0 if its index i corresponds to the one in that binary vector and the one is then left-shifted by one bit (its control signal is denoted by **MC2**). In comparison, the **PE** of **ePIBMA** requires one more layer of multiplexer than that of **PIBMA**, but is identical otherwise, where the control signal **MC3** represents the Boolean operation “ $L_B < t - 1$ ”. Our analysis will focus on **ePIBMA** as it is readily pipelined to the **ecSEE**. Note that each **PE** of **ePIBMA** contains 2 multipliers, 1 adder, 3 multiplexers, and 2 registers. Ignoring the negligible complexity of control unit, the **ePIBMA** architecture requires $2t + 1$ adders, $4t + 2$ multipliers, $4t + 2$ registers, and $6t + 3$ multiplexers. It is plausible to claim the critical path delay is caused by one multiplication and one addition in updating $\hat{\Omega}^{(r)}(x)$, i.e.,

$$T_{\text{crit-path}} = T_{\text{mult}} + T_{\text{add}}. \quad (20)$$

The multiplex operations are used in updating $\hat{\Theta}^{(r)}(x)$ and thus do not affect the critical path delay. In comparison with the **eIBMA** architecture, the **ePIBMA** architecture significantly shortens the critical path delay from originally $2T_{\text{mult}} + (1 + \lceil \log_2(t + 1) \rceil) \cdot T_{\text{add}}$ to the minimum, $T_{\text{mult}} + T_{\text{add}}$, which is necessary for the error locator update.

Table I lists implementation costs and critical path delays of the existing algorithms that solve the key equation (4) in $2t$ clock cycles (i.e., their outputs are directly pipelined to the processor of the Chien search and error evaluation). As a matter of fact, if longer delay is tolerated, then essentially each **PE** of a systolic architecture can be repeatedly used and thus the hardware complexity can be reduced to a fraction [12]. For instance, in [5], the extended Euclidean algorithm (**EEA**) is implemented in a folded fashion such that the hardware complexity is reduced to roughly a quarter but the number of clocks is increased by five times; in [21], the **IBMA** is implemented in serial so that only 3 multipliers are employed.

eIBMA and **IBMA**—[10] have similar irregular architecture, i.e., they are both composed of discrepancy computation block and error locator update block, and thus the critical path delay suffers from the cascade of the two. Clearly, the **eIBMA** requires the minimum hardware, but has more than twice the critical path delay, comparing to the parallel algorithms **ePIBMA**, **RiBMA**—[11], **EEA**—[4]. **ePIBMA** and **RiBMA** have similar regular architectures, both employing inversionless B-M iteration. The **PEs** in the two architectures are otherwise identical except that the proposed one has two additional multiplexers. The prominent difference is that the proposed **ePIBMA** employs t less **PEs** than **RiBMA**. In addition, it is worth noting that **RiBMA** is not scalable, as the beginning position of its output, $\hat{\Omega}(x)$, is dependent of t , while **ePIBMA** always starts from the first **PE** regardless of t . This feature renders **ePIBMA** desirable for configurable design, a common-practice in industry. **EEA** also exhibits regular and scalable architecture, while it exploits twice as much as the hardware of the proposed **ePIBMA**. The decoder architecture building on the Degree Computationless Modified Euclid’s Algorithm (**DCMEA**)—[9] is an enhanced version of **EEA** and yields an architecture with similar performance as **RiBMA**. Two-folded **ePIBMA** micro-architecture is depicted in Fig. 7, in which the additional control signal **CC** denotes the clock counter. It is worth noting that the two-folded **ePIBMA** compares favorably to the **eIBMA**, in both hardware complexity and overall latency as product of cycle number and clock frequency.

C. **ePIBMA** as LFSR Encoder

In this subsection, we demonstrate that **ePIBMA** can effectively operate as an LFSR encoder. We observe that $\hat{\Omega}(x)$ forms an LFSR array. However, its feedback comes from the constant term, whereas the LFSR encoder uses the highest term as feedback. To this end, it suffices to reverse the representation of $\hat{\Omega}(x)$ and set generator polynomial terms reversely to $\hat{\Theta}(x)$. The detailed algorithmic procedure is described as follows.

ePIBMA as LFSR Encoder

- Input: $[D_{k-1}, D_{k-2}, \dots, D_1, D_0]$
- Initialization: $\hat{\Theta}^{(0)}(x) = G_{2t-1} + G_{2t-2}x + \dots + G_1x^{2t-2} + G_0x^{2t-1}$, $\hat{\Omega}^{(0)}(x) = 0$, $\gamma = 1$
- For $r = 0, 1, 2, \dots, k - 2, k - 1$, do:
 - Compute “discrepancy” $\Delta^{(r)} = \hat{\Omega}_0^{(r)} + D_{k-1-r}$
 - Compute $\hat{\Omega}^{(r+1)}(x) = \gamma \mathcal{L}_1 \hat{\Omega}^{(r)}(x) - \Delta^{(r)} \hat{\Theta}^{(r)}(x)$
 - Set $\hat{\Theta}^{(r+1)}(x) = \hat{\Theta}^{(r)}(x)$
- Output: $\Psi(x) = \hat{\Omega}_0^{(k)} x^{2t-1} + \hat{\Omega}_1^{(k)} x^{2t-2} + \dots + \hat{\Omega}_{2t-2}^{(k)} x + \hat{\Omega}_{2t-1}^{(k)}$

We note the above algorithmic procedure enables to utilize the **PE** of **ePIBMA** without any modification, except for a slight change in the control logic. In order to disable **MC2** which dynamically sets $\hat{\Theta}_{2t-r-2}^{(r+1)} = 0$, it suffices to broadcast the all-zero vector, instead of a unit vector. In particular, this

TABLE I
ERROR-ONLY DECODER COMPARISONS OF HARDWARE COMPLEXITY AND CRITICAL PATH

Architecture	Adders	Multipliers	Latches	Muxes	Clocks	$T_{\text{crit-path}}$
eIBMA	$2t$	$3t + 2$	$3t + 3$	$t + 1$	$2t$	$2T_{\text{mult}} + \lceil \log_2(2t + 2) \rceil T_{\text{add}}$
ePIBMA	$2t + 1$	$4t + 2$	$4t + 2$	$6t + 3$	$2t$	$T_{\text{mult}} + T_{\text{add}}$
ePIBMA(2-fold)	$t + 1$	$2t + 2$	$6t + 6$	$5t + 5$	$4t$	$T_{\text{mult}} + T_{\text{add}} + T_{\text{mux}}$
IBMA –[10]	$2t + 1$	$3t + 3$	$4t + 2$	$t + 1$	$2t$	$2T_{\text{mult}} + \lceil \log_2(2t + 2) \rceil T_{\text{add}}$
RiBMA –[11]	$3t + 1$	$6t + 2$	$6t + 2$	$3t + 1$	$2t$	$T_{\text{mult}} + T_{\text{add}}$
EEA –[4]	$4t + 2$	$8t + 8$	$4t + 4$	$8t + 8$	$2t$	$T_{\text{mult}} + T_{\text{add}} + T_{\text{mux}}$
DCMEA –[9]	$3t$	$6t$	$6t$	$2t - 1$	$2t$	$T_{\text{mult}} + T_{\text{add}} + T_{\text{mux}}$

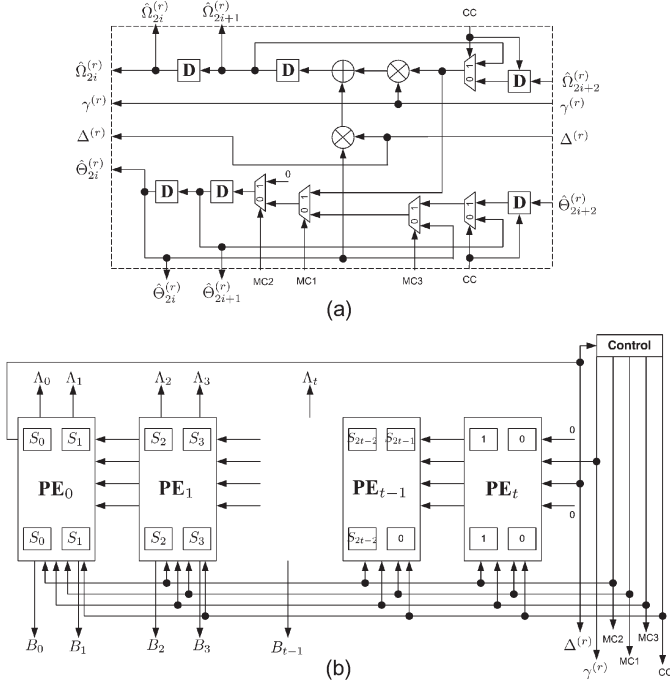


Fig. 7. The systolic two-folded **ePIBMA** architecture. (a) The **PE** processor. (b) two-folded **ePIBMA** block diagram.

feature saves large encoder area in a scalable setup where $2t$ full-multipliers must be used to multiply with G_i , $i = 0, 1, \dots, 2t - 1$.

V. PARALLEL INVERSIONLESS BLAHUT ALGORITHMS AND THEIR SYSTOLIC ARCHITECTURES

Sarwate and Shanbhag [11] presented a systolic **RiBMA** architecture for error-only decoding. Through simultaneously updating of the complementary error evaluator polynomial and error locator polynomial, the discrepancy values are always readily available from the (same) first location and thus the critical path delay is significantly shortened by avoiding computing discrepancy value on the critical path [11].

In this section we extend this parallel mechanism to the error-erasure decoding through transforming **riBA**. We first observe that the complementary error-erasure evaluator polynomial can be produced iteratively, analogous to the complementary error evaluator polynomial in **RiBMA** in [11]. Its algorithmic procedure is listed below, where the shift operator $\mathcal{L}_1$ is defined in (16).

Iteratively Computing Complementary Error-Erasure Evaluator Polynomial

- Input: $\mathbf{S} = [S_0, S_1, S_2, \dots, S_{2t-1}], [\alpha_0, \alpha_1, \dots, \alpha_{\rho-1}]$
- Initialization: $\hat{\Omega}^{(0)}(x) = \hat{\Theta}^{(0)}(x) = S_0 + S_1x + \dots + S_{2t-1}x^{2t-1}$, $\gamma^{(0)} = 1$, $L_B^{(0)} = L_\Lambda^{(0)} = 0$
- For $r = 0, 1, 2, \dots, 2t - 1$, do:
 - Set $\Delta^{(r)} \leftarrow \begin{cases} \alpha_r, & \text{if } r < \rho \\ \hat{\Omega}_0^{(r)}, & \text{otherwise} \end{cases}$
 - $\hat{\Omega}^{(r+1)}(x) = \gamma^{(r)} \cdot [\mathcal{L}_1 \hat{\Omega}^{(r)}](x) - \Delta^{(r)} \cdot \hat{\Theta}^{(r)}(x)$
 - if $r < \rho$
 - * Set $\hat{\Theta}^{(r+1)}(x) \leftarrow \hat{\Omega}^{(r+1)}(x)$
 - * Set $\gamma^{(r+1)} \leftarrow \gamma^{(r)}$
 - * Set $L_B^{(r+1)} \leftarrow r + 1, L_\Lambda^{(r+1)} \leftarrow r + 1$
 - Else if $\Delta^{(r)} \neq 0$ & $L_\Lambda^{(r)} \leq L_B^{(r)}$
 - * Set $\hat{\Theta}^{(r+1)}(x) \leftarrow [\mathcal{L}_1 \hat{\Omega}^{(r)}](x)$
 - * Set $\gamma^{(r+1)} \leftarrow \Delta^{(r)}$
 - * Set $L_\Lambda^{(r+1)} \leftarrow L_B^{(r)} + 1, L_B^{(r+1)} \leftarrow L_\Lambda^{(r)}$
 - Else
 - * Set $\hat{\Theta}^{(r+1)}(x) \leftarrow \hat{\Theta}^{(r)}(x)$
 - * Set $\gamma^{(r+1)} \leftarrow \gamma^{(r)}$
 - * Set $L_B^{(r+1)} \leftarrow L_B^{(r)} + 1, L_\Lambda^{(r+1)} \leftarrow L_\Lambda^{(r)}$
- Output: $\hat{\Omega} = [\hat{\Omega}_0^{(2t)}, \hat{\Omega}_1^{(2t)}, \hat{\Omega}_2^{(2t)}, \dots, \hat{\Omega}_{2t-1}^{(2t)}], L_\Lambda$

A key advantage of the above algorithm is that the discrepancy is always readily available without extra computation

$$\Delta^{(r)} = \begin{cases} \alpha_r, & \text{if } r < \rho \\ \hat{\Omega}_0^{(r)}, & \text{otherwise.} \end{cases}$$

The next lemma characterizes the above algorithm.

Lemma 7: (i). $\hat{\Omega}^{(r)}(x)$ and $\hat{\Theta}^{(r)}(x)$ is a left-shifted polynomial of $\Lambda^{(r)}(x)S(x)$ and $B^{(r)}(x)S(x)$ respectively, i.e.,

$$\begin{aligned} \hat{\Omega}^{(r)}(x) &= [\mathcal{L}_r(\Lambda^{(r)}S)](x), \\ \hat{\Theta}^{(r)}(x) &= [\mathcal{L}_r(B^{(r)}S)](x), \end{aligned}$$

where $\Lambda^{(r)}(x)$ and $B^{(r)}(x)$ denote the error-erasure locator polynomial and its auxiliary polynomial generated at the r -th iteration of **riBA**.

(ii). The degrees of $\hat{\Omega}^{(r)}(x)$ and $\hat{\Theta}^{(r)}(x)$ satisfy

$$\deg(\hat{\Omega}^{(r)}(x)) < 4t - r, \quad \deg(\hat{\Theta}^{(r)}(x)) < 4t - r$$

for $r = 0, 1, 2, \dots, 2t - 1$.

(iii). Let $\Omega(x)$ be the error-erasure evaluator polynomial as defined in (4), then

$$\Omega(x) + x^{2t} \cdot \hat{\Omega}(x) = \Lambda(x)S(x). \quad (21)$$

Proof: Part (i) follows trivially by induction. Part (ii) follows from the observation

$$r + \deg(\hat{\Omega}^{(r)}(x)) = \deg(\Lambda^{(r)}S)(x) < r + 2t \leq 4t$$

$$r + \deg(\hat{\Theta}^{(r)}(x)) = \deg(B^{(r)}S)(x) < r + 2t \leq 4t.$$

(iii) follows from (i) and (4). $\square\square$

Part (iii) clarifies the terminology “complementary error-erasure evaluator polynomial” for $\hat{\Omega}(x)$. Let X^{-1} be a root of $\Lambda(x)$, then (21) reduces to

$$\Omega(X^{-1}) = X^{-2t} \hat{\Omega}(X^{-1}).$$

Thus we can compute error magnitudes through a modified Forney formula as follows

$$Y_i = \frac{X_i^{-1} \Omega(X_i^{-1})}{\Lambda_{\text{odd}}(X_i^{-1})} = \frac{X_i^{-2t-1} \hat{\Omega}(X_i^{-1})}{\Lambda_{\text{odd}}(X_i^{-1})}, \quad (22)$$

where X_i^{-1} is a root of $\Lambda(x)$.

The next lemma casts an early stopping rule when the number of errors is smaller than the given capability $t - \lceil \frac{\rho}{2} \rceil$.

Lemma 8: (i). If $B^{(r)}(x)$ has length $L_B^{(r)} = t + \lceil \frac{\rho}{2} \rceil$, then, either more than $t - \lceil \frac{\rho}{2} \rceil$ errors have occurred (i.e., uncorrectable), or $\Lambda^{(r)}(x)$ contains all error-erasure locators as inverse of its roots.

(ii). Suppose $\Lambda^{(r)}(x)$ contains all error-erasure locators as inverse of its roots. Define $\Lambda^{*(r)}(x) = x^{2t-r} \Lambda^{(r)}(x)$. Then, an error magnitude can be alternatively expressed as

$$Y_i = \frac{X_i^{-2t-1} \hat{\Omega}^{(r)}(X_i^{-1})}{\Lambda_{\text{odd}}^{*(r)}(X_i^{-1})}, \quad (23)$$

where X_i^{-1} is a root of $\Lambda^{(r)}(x)$.

Proof: (i). If the discrepancy values $\Delta^{(r+1)}, \Delta^{(r+2)}, \dots, \Delta^{(2t-1)}$, are all zeros, then $\Lambda^{(2t)}(x) = (\gamma^{(r)})^{2t-r} \Lambda^{(r)}(x)$ is either correctable or uncorrectable. Otherwise, let $\Delta^{(s)}$ ($s > r$) be the first nonzero discrepancy, then $\Lambda^{(s)}(x) = (\gamma^{(r)})^{s-r} \Lambda^{(r)}(x) + \Delta^{(s)} x^{s-r} B^{(r)}(x)$ exhibits a length greater than the error correction capability $t + \lceil \frac{\rho}{2} \rceil$, so will be $\Lambda^{(2t)}(x)$. Thus $\Lambda^{(2t)}(x)$ must be uncorrectable.

(ii). By assumption, we observe that $\Delta^{(r+1)}, \Delta^{(r+2)}, \dots, \Delta^{(2t-1)}$, are all zeros. This reveals a scaling relation

$$\Lambda^{(2t)}(x) = (\gamma^{(r)})^{2t-r} \Lambda^{(r)}(x).$$

We further note

$$\begin{aligned} \hat{\Omega}^{(r)}(x) &\triangleq \left[\mathcal{L}_r \left(\Lambda^{(r)} S \right) \right] (x) \\ &= x^{2t-r} \left[\mathcal{L}_{2t} \left(\Lambda^{(r)} S \right) \right] (x) \\ &= x^{2t-r} \left(\gamma^{(r)} \right)^{-(2t-r)} \hat{\Omega}^{(2t)}(x) \end{aligned}$$

where the second equality is due to that

$$\deg(\Lambda^{(r)} S)(x) \bmod x^{2t} < r.$$

The alternative Forney formula follows

$$\begin{aligned} \frac{\hat{\Omega}(X_i^{-1})}{\Lambda_{\text{odd}}(X_i^{-1})} &= \frac{\hat{\Omega}^{(2t)}(X_i^{-1})}{\Lambda_{\text{odd}}^{(2t)}(X_i^{-1})} \\ &= \frac{(\gamma^{(r)})^{2t-r} X_i^{-(2t-r)} \hat{\Omega}^{(r)}(X_i^{-1})}{(\gamma^{(r)})^{2t-r} \Lambda_{\text{odd}}^{(r)}(X_i^{-1})} \\ &= \frac{\hat{\Omega}^{(r)}(X_i^{-1})}{\Lambda_{\text{odd}}^{*(r)}(X_i^{-1})}. \end{aligned}$$

$\square\square$

We are now ready to merge $\Lambda(x)$ and $B(x)$ into $\hat{\Omega}(x)$ and $\hat{\Theta}(x)$, respectively, and unify the above algorithm into a homogeneous Parallel Inversionless Blahut Algorithm (PIBA) and justify it thereafter.

Parallel Inversionless Blahut Algorithm (PIBA)

- Inputs: $\mathbf{S} = [S_0, S_1, S_2, \dots, S_{2t-1}]$, $[\alpha_0, \alpha_1, \dots, \alpha_{\rho-1}]$
 - Initialization: $\hat{\Omega}^{(0)}(x) = \hat{\Theta}^{(0)}(x) = S_0 + S_1 x + \dots + S_{2t-1} x^{2t-1} + x^{4t}$, $\gamma^{(0)} = 1$, $L_B^{(0)} = L_{\Lambda}^{(0)} = 0$
 - For $r = 0, 1, 2, \dots, 2t - 1$, do:
 - Set $\Delta^{(r)} \leftarrow \begin{cases} \alpha_r, & \text{if } r < \rho \\ \hat{\Omega}_0^{(r)}, & \text{otherwise} \end{cases}$
 - $\hat{\Omega}^{(r+1)}(x) = \gamma^{(r)} \cdot [\mathcal{L}_1 \hat{\Omega}^{(r)}](x) - \Delta^{(r)} \cdot \hat{\Theta}^{(r)}(x)$
 - If $r < \rho$
 - * Set $\hat{\Theta}^{(r+1)}(x) \leftarrow \hat{\Omega}^{(r+1)}(x)$
 - * Set $\gamma^{(r+1)} \leftarrow \gamma^{(r)}$
 - * Set $L_B^{(r+1)} \leftarrow r + 1$, $L_{\Lambda}^{(r+1)} \leftarrow r + 1$
 - Else if $\Delta^{(r)} \neq 0$ & $L_{\Lambda}^{(r)} \leq L_B^{(r)}$
 - * Set $\hat{\Theta}^{(r+1)}(x) \leftarrow [\mathcal{L}_1 \hat{\Omega}^{(r)}](x)$
 - * Set $\gamma^{(r+1)} \leftarrow \Delta^{(r)}$
 - * Set $L_{\Lambda}^{(r+1)} \leftarrow L_B^{(r)} + 1$, $L_B^{(r+1)} \leftarrow L_{\Lambda}^{(r)}$
 - Else
 - * Set $\hat{\Theta}^{(r+1)}(x) \leftarrow \hat{\Theta}^{(r)}(x)$
 - * Set $\gamma^{(r+1)} \leftarrow \gamma^{(r)}$
 - * Set $L_B^{(r+1)} \leftarrow L_B^{(r)} + 1$, $L_{\Lambda}^{(r+1)} \leftarrow L_{\Lambda}^{(r)}$
 - If $L_B^{(r+1)} = t + \lceil \frac{\rho}{2} \rceil$, then break For-loop
 - Outputs: $\hat{\Omega} = [\hat{\Omega}_0, \hat{\Omega}_1, \dots, \hat{\Omega}_{2t-1}]$, $\Lambda = [\hat{\Omega}_{2t}, \hat{\Omega}_{2t+1}, \dots, \hat{\Omega}_{4t}]$, L_{Λ}
-

We now show by contradiction that $L_B^{(r+1)} \geq t + \lceil \frac{\rho}{2} \rceil$ does not occur in the case $\Delta^{(r)} \neq 0$ & $L_\Lambda^{(r)} \leq L_B^{(r)}$. Otherwise, it holds $L_\Lambda^{(r)} \geq t + \lceil \frac{\rho}{2} \rceil$. The condition $L_\Lambda^{(r)} \leq L_B^{(r)}$ further suggests $L_B^{(r)} \geq t + \lceil \frac{\rho}{2} \rceil$. This contradicts the assumption that $r + 1$ is the first clock number to satisfy $L_B^{(r+1)} \geq t + \lceil \frac{\rho}{2} \rceil$.

The following lemma asserts that the concatenated two polynomials $\Lambda(x)$ and $\hat{\Omega}(x)$ (analogously $B(x)$ and $\hat{\Theta}(x)$) operate independently and $\Lambda^{(0)}(x) = x^{4t}$ is the minimum degree to guarantee non-overlapping.

Lemma 9: In PIBA, the concatenation of two polynomials $\Lambda(x)$ and $\hat{\Omega}(x)$ (analogously $B(x)$ and $\hat{\Theta}(x)$) are guaranteed to be non-overlapping and tight, i.e.,

(i). at any iteration r , $0 \leq r \leq 2t$, the degree of $\hat{\Omega}^{(r)}(x)$ is smaller than the starting power of $\Lambda^{(r)}(x)$;

(ii). there exist scenarios where the degree of $\hat{\Omega}^{(r)}(x)$ is exactly one less than the starting power of $\Lambda^{(r)}(x)$.

Proof: (i). By Lemma 7.4(ii), $\deg(\hat{\Omega}^{(r)}(x)) < 4t - r$. On the other hand, $\Lambda^{(0)}(x) = x^{4t}$, i.e., the starting power of $\Lambda^{(r)}(x)$ is $4t$ at $r = 0$ and decreases by one for each iteration due to left shift operation, therefore, $\Lambda^{(r)}(x)$ starts exactly at x^{4t-r} .

(ii). When there are $2t$ erasures (i.e., erasure-only decoding) and $r = 2t$ (i.e., at the end of last iteration), we have the following equality on the degree of product polynomial $\Lambda(x)S(x)$

$$2t + \deg \left(\hat{\Omega}^{(2t)}(x) \right) = 2t + \deg \left(\hat{\Theta}^{(2t)}(x) \right) = 4t - 1,$$

where $2t$ on the left side is due to $2t$ left shifts. Note $\Lambda^{(2t)}(x)$ starts exactly at the power $2t$, precisely one more than $\deg(\hat{\Omega}^{(2t)}(x))$. $\square\square$

Note when early stopping rule is provoked at the iteration r , the outputs are precisely

$$\hat{\Omega}(x) = \hat{\Omega}^{(r+1)}(x), \quad \Lambda(x) = \Lambda^{*(r+1)}(x).$$

Therefore, the original expression for error magnitudes (22) applies unanimously. The systolic architecture of **PIBA** is illustrated in Fig. 8, which contains an array of $4t + 1$ homogeneous systolic processor elements (**PE**). Two control signals are broadcast. The erasure control-signal **ERS** represents the Boolean operation “ $r < \rho$ ”. The minimization control-signal **MIN** stands for “ $\hat{\omega}_0^{(r)} \neq 0$ and $L_\Lambda^{(r)} \leq L_B^{(r)}$ ”. Ignoring the negligible complexity of the control unit, the proposed architecture requires $4t + 1$ adders, $8t + 2$ multipliers, $8t + 2$ registers, and $8t + 2$ multiplexers. The critical path starts from the multiplexer for $\Delta^{(r)}$ and ends with the multiplexer for $\hat{\Theta}^{(r+1)}(x)$, passing through one multiplier, one adder, and two multiplexers, namely,

$$T_{\text{crit-path}} = T_{\text{mult}} + T_{\text{add}} + 2T_{\text{mux}}. \quad (24)$$

Retiming can be deployed to shorten the critical path. Specifically, it is noted that the multiplex for $\Delta^{(r)}$ is independent of the multiplex for $\hat{\Theta}^{(r+1)}(x)$. It is therefore desirable to retime the register $\hat{\Theta}(x)$ to parallelize the two multiplex operations. The corresponding architecture, called **Retimed Parallel Inversionless Blahut Algorithm (rPIBA)**, is depicted in Fig. 9. It is worth noting that the erasure control-signal **ERS** now represents the Boolean operation “ $r \leq \rho$ ”, different from that of **PIBA**.

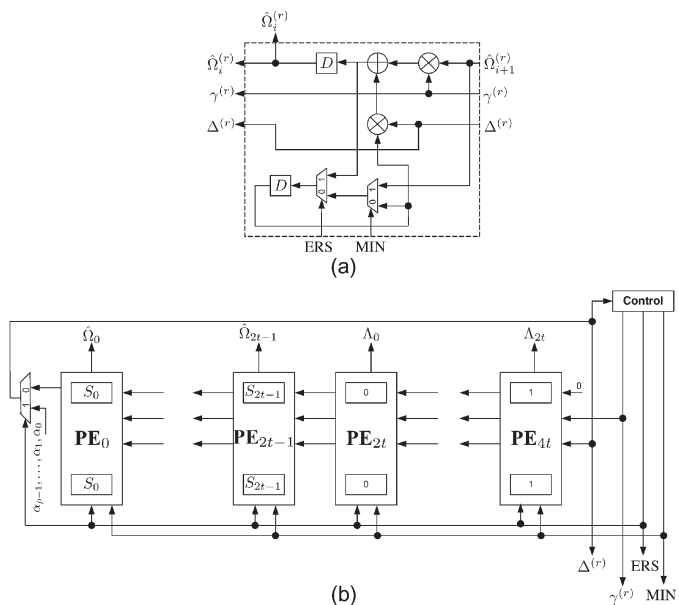


Fig. 8. **PIBA** systolic architecture. (a) The **PE** processor. (b) The block diagram.

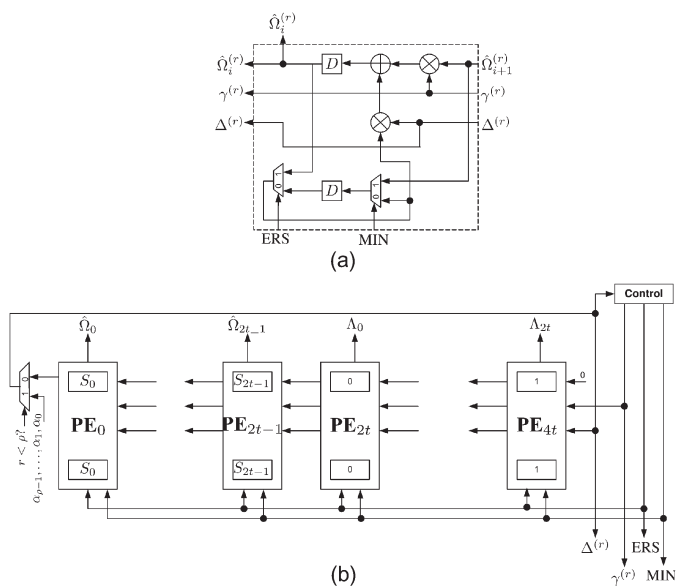


Fig. 9. **rPIBA** systolic architecture. (a) The **PE** processor. (b) The block diagram.

It is instrumental to compare with the similar systolic architecture given in [22], which follows a mixture of the extended Euclidean algorithm and the inversionless Blahut algorithm (for notation conciseness, we call it **EEIBA**). The two architectures share the short latency of $2t$ cycles. **EEIBA** is marginally inferior, by having a critical path delay of one multiplication, one addition and two multiplexes (vs. one multiplex), and utilizing $4t + 4$ systolic cells (vs. $4t + 1$ cells). The detailed complexities and latencies of the both are listed in Table II. From algorithmic perspective, there lie four evident differences. The two architectures employ very different initialization, $\hat{\Omega}^{(0)}(x) = \hat{\Theta}^{(0)}(x) = S(x) + x^{4t}$ in the **rPIBA**, in contrast to $\Omega^{(0)}(x) = x^{2t} + S(x) + x^{4t+1}$ and $\Theta^{(0)}(x) = 0$ in **EEIBA**. The

TABLE II
DECODER COMPARISONS OF HARDWARE COMPLEXITY AND CRITICAL PATH

Architectures	Error Type	Adders	Multipliers	Latches	Muxes	Clocks	$T_{\text{crit-path}}$
eIBMA	Error-Only	$2t$	$3t + 2$	$2t + 2$	$t + 1$	$2t$	$2T_{\text{mult}} + \lceil \log_2(2t + 2) \rceil T_{\text{add}}$
ePIBMA	Error-Only	$2t + 1$	$4t + 2$	$4t + 2$	$6t + 3$	$4t$	$T_{\text{mult}} + T_{\text{add}}$
rIBA	Error-Erasure	$4t$	$6t + 2$	$4t + 2$	$2t + 1$	$4t$	$2T_{\text{mult}} + \lceil \log_2(4t) \rceil T_{\text{add}} + 2T_{\text{mux}}$
rPIBA	Error-Erasure	$4t + 1$	$8t + 2$	$8t + 2$	$8t + 2$	$2t$	$T_{\text{mult}} + T_{\text{add}} + T_{\text{mux}}$
rPIBA(2-fold)	Error-Erasure	$2t + 1$	$4t + 2$	$10t + 5$	$6t + 3$	$4t$	$T_{\text{mult}} + T_{\text{add}} + T_{\text{mux}}$
SPIBA	Error-Erasure	$2t + 1$	$4t + 2$	$4t + 2$	$4t + 2$	$4t$	$T_{\text{mult}} + T_{\text{add}} + T_{\text{mux}}$
UIBA	Unified	$2t + 1$	$4t + 2$	$4t + 2$	$6t + 3$	$4t/2t$	$T_{\text{mult}} + T_{\text{add}} + T_{\text{mux}}$
EEIBA – [22]	Error-Erasure	$4t + 4$	$8t + 8$	$8t + 8$	$8t + 8$	$2t$	$T_{\text{mult}} + T_{\text{add}} + 2T_{\text{mux}}$

proposed **rPIBA** always produces the complementary error-erasure evaluator polynomial at same starting location for a given t ; whereas **EEIBA** produces error-erasure evaluator polynomial at dynamical starting locations, rendering it hard for parallel loading. The proposed **rPIBA** exhibits simpler control unit design based on the proposed algorithmic optimization, which may well become the critical delay if not carefully designed. Lastly, **rPIBA** enforces an early stopping rule to save power dynamically under the predominant condition that the number of errors is below the given correction capability.

We observe that **rPIBA** is naturally divided into two disjoint parts, the left $2t$ cells to iteratively compute $\hat{\Omega}(x)$ whereas the right $2t + 1$ cells iteratively compute $\Lambda(x)$. Thus, by allowing twice of latency, i.e., $4t$ cycles, we can effectively halve the complexity, as shown below.

Split Parallel Inversionless Blahut Algorithm (SPIBA)

I. Compute Complementary Error-Erasure Evaluator Polynomial

- Inputs: $\mathbf{S} = [S_0, S_1, \dots, S_{2t-1}], [\alpha_0, \alpha_1, \dots, \alpha_{\rho-1}]$
- Initialization: $\hat{\Omega}^{(0)}(x) = \hat{\Theta}^{(0)}(x) = S_0 + S_1x + \dots + S_{2t-1}x^{2t-1}$, $\gamma^{(0)} = 1$, $L_B^{(0)} = L_\Lambda^{(0)} = 0$
- For $r = 0, 1, 2, \dots, 2t - 1$, do:
 - Set $\Delta^{(r)} \leftarrow \begin{cases} \alpha_r, & \text{if } r < \rho \\ \hat{\Omega}_0^{(r)}, & \text{otherwise} \end{cases}$
 - If $r \leq \rho$, then set $\hat{\Theta}^{(r)}(x) \leftarrow \hat{\Omega}^{(r)}(x)$ (retimed)
 - $\hat{\Omega}^{(r+1)}(x) = \gamma^{(r)} \cdot [\mathcal{L}_1 \hat{\Omega}^{(r)}](x) - \Delta^{(r)} \cdot \hat{\Theta}^{(r)}(x)$
 - $\hat{\Theta}^{(r+1)}(x) \leftarrow \begin{cases} [\mathcal{L}_1 \hat{\Omega}^{(r)}](x), & \text{if } \Delta^{(r)} \neq 0 \text{ \& } L_\Lambda^{(r)} \leq L_B^{(r)} \\ \hat{\Theta}^{(r)}(x), & \text{else} \end{cases}$
 - If $r < \rho$
 - Set $\gamma^{(r+1)} \leftarrow \gamma^{(r)}$
 - Set $L_B^{(r+1)} \leftarrow r + 1$, $L_\Lambda^{(r+1)} \leftarrow r + 1$
 - Else if $\Delta^{(r)} \neq 0 \text{ \& } L_\Lambda^{(r)} \leq L_B^{(r)}$
 - Set $\gamma^{(r+1)} \leftarrow \Delta^{(r)}$
 - Set $L_\Lambda^{(r+1)} \leftarrow L_B^{(r)} + 1$, $L_B^{(r+1)} \leftarrow L_\Lambda^{(r)}$
 - Else
 - Set $\gamma^{(r+1)} \leftarrow \gamma^{(r)}$
 - Set $L_B^{(r+1)} \leftarrow L_B^{(r)} + 1$, $L_\Lambda^{(r+1)} \leftarrow L_\Lambda^{(r)}$
 - If $L_B^{(r+1)} = t + \lceil \frac{\rho}{2} \rceil$, then break For-loop
- Outputs: $\hat{\Omega} = [\hat{\Omega}_0, \hat{\Omega}_1, \dots, \hat{\Omega}_{2t-1}]$, $\Delta = [\alpha_0, \alpha_1, \dots, \alpha_{\rho-1}, \Delta^{(\rho)}, \Delta^{(\rho+1)}, \dots, \Delta^{(2t-1)}]$

II. Compute Error-Erasure Locator Polynomial

- Inputs: $\Delta = [\alpha_0, \alpha_1, \dots, \alpha_{\rho-1}, \Delta^{(\rho)}, \Delta^{(\rho+1)}, \dots, \Delta^{(2t-1)}]$
- Initialization: $\hat{\Omega}^{(0)}(x) = \hat{\Theta}^{(0)}(x) = x^{2t}$, $\gamma^{(0)} = 1$, $L_B^{(0)} = L_\Lambda^{(0)} = 0$
- For $r = 0, 1, 2, \dots, 2t - 1$, do:
 - If $r \leq \rho$, then set $\hat{\Theta}^{(r)}(x) \leftarrow \hat{\Omega}^{(r)}(x)$ (retimed)
 - $\hat{\Omega}^{(r+1)}(x) = \gamma^{(r)} \cdot [\mathcal{L}_1 \hat{\Omega}^{(r)}](x) - \Delta^{(r)} \cdot \hat{\Theta}^{(r)}(x)$
 - $\hat{\Theta}^{(r+1)}(x) \leftarrow \begin{cases} [\mathcal{L}_1 \hat{\Omega}^{(r)}](x), & \text{if } \Delta^{(r)} \neq 0 \text{ \& } L_\Lambda^{(r)} \leq L_B^{(r)} \\ \hat{\Theta}^{(r)}(x), & \text{else} \end{cases}$
 - If $r < \rho$
 - Set $\gamma^{(r+1)} \leftarrow \gamma^{(r)}$
 - Set $L_B^{(r+1)} \leftarrow r + 1$, $L_\Lambda^{(r+1)} \leftarrow r + 1$
 - Else if $\Delta^{(r)} \neq 0 \text{ \& } L_\Lambda^{(r)} \leq L_B^{(r)}$
 - Set $\gamma^{(r+1)} \leftarrow \Delta^{(r)}$
 - Set $L_\Lambda^{(r+1)} \leftarrow L_B^{(r)} + 1$, $L_B^{(r+1)} \leftarrow L_\Lambda^{(r)}$
 - Else
 - Set $\gamma^{(r+1)} \leftarrow \gamma^{(r)}$
 - Set $L_B^{(r+1)} \leftarrow L_B^{(r)} + 1$, $L_\Lambda^{(r+1)} \leftarrow L_\Lambda^{(r)}$
 - If $L_B^{(r+1)} = t + \lceil \frac{\rho}{2} \rceil$, then break For-loop
- Outputs: $\Lambda = [\hat{\Omega}_0, \hat{\Omega}_1, \hat{\Omega}_2, \dots, \hat{\Omega}_{2t}], L_\Lambda$

When the algorithm is terminated early, the remaining discrepancies are all zeros. The systolic architecture of Part I is drawn in Fig. 10. There are two minor differences between Parts I and II, neither affecting circuit design. One is their register initialization, namely, $\hat{\Omega}^{(0)}(x) = \hat{\Theta}^{(0)}(x) = S(x)$ in Part I, whereas $\hat{\Omega}^{(0)}(x) = \hat{\Theta}^{(0)}(x) = x^{2t}$ in Part II. The other is the multiplex operation to choose discrepancy $\Delta^{(r)}$. In Part I, the multiplexer is controlled by “ $r < \rho$ ”, whereas it is set to 1 in Part II (i.e., $\Delta^{(r)}$ always chooses the input sequence, $\alpha_0, \alpha_1, \dots, \alpha_{\rho-1}, \Delta^{(\rho)}, \Delta^{(\rho+1)}, \dots, \Delta^{(2t-1)}$). Also note that erasure register array has size $2t$ (for the worst-case). It is therefore convenient to store discrepancy vector in the erasure register array. To shorten the latency of the comparison $r < \rho$, it is beneficial to introduce a biased counter, $\tilde{r} \triangleq r - \rho$ with initialization of $-\rho$. Then, the condition $r < \rho$ is equivalent to $\tilde{r} < 0$, which becomes simply to compare its sign bit. Analogously, the comparison $r \leq \rho$ (represented by the control signal **ERS**) is simplified by introducing another biased counter $\hat{r} \triangleq r - \rho - 1$ with initialization of $-\rho - 1$. Then, the condition $r \leq \rho$ is equivalent to $\hat{r} < 0$.

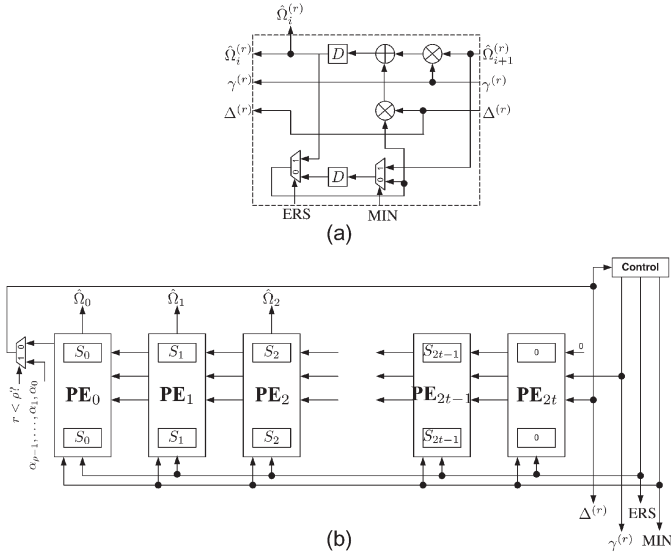


Fig. 10. **SPIBA** systolic architecture. (a) The **PE** processor. (b) The block diagram.

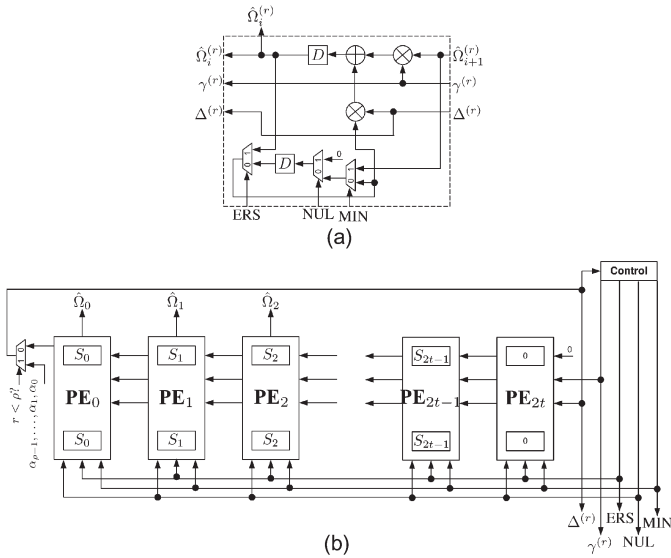


Fig. 11. The **UIBA** systolic architecture. (a) **PE** processor. (b) The block diagram.

VI. A UNIFIED DECODER ARCHITECTURE

In Section IV, we presented a systolic architecture **ePIBMA** for the error-only decoding of RS codes. It employs $2t+1$ systolic cells, matching to **SPIBA** in preceding section. Note error-only decoding is a special case of error-erasure decoding, therefore **SPIBA** readily handles both cases. However, this is inefficient for error-only decoding, because **SPIBA** runs $4t$ cycles while **ePIBMA** runs $2t$ cycles. In this section, we present a unified architecture that is efficient for both error-only and error-erasure decoding. We observe that limiting the length of $B(x)$ to $t-1$, as enforced in **ePIBMA**, is unnecessary, since $\hat{\Omega}(x)$ may have degree up to $2t-1$ in error-erasure decoding. Therefore, it suffices to incorporate **PIBMA** instead.

We also note that the early termination rule given in **SPIBA** is inapplicable for error-only decoding, this is because Δ_0 read from the first cell is zero otherwise, rendering the generalized Horiguchi-Koetter formula invalid. In order to achieve the efficiency of **PIBMA** for error-only decoding, it is essential to incorporate the nullification operation, $\hat{\Omega}_{2t-r-2}^{(r+1)} \leftarrow 0, 0 \leq r < 2t$. In **PIBMA**, the nullification is controlled by the signal **NUL** which represents the Boolean operation “if the index corresponds to the (only) one in the unit vector”. The sequential reset is implemented through transmitting a $2t$ -dimensional unit vector which is cyclically shifted in each iteration and set a register to zero if the corresponding bit of the vector is 1. To disable the sequential reset for error-erasure decoding, it suffices to transmit instead an all-zero vector, which is cost free.

The dedicated architecture **UIBA** is drawn in Fig. 11, in which the retiming technique is deployed to reduce the critical path delay. Its critical path delay has a multiplication, an addition, and a multiplex, along with broadcast, very close to that of the error-only decoder architecture **ePIBMA**. The detailed algorithm is described in the following pseudo code.

Unified Parallel Inversionless Blahut Algorithm (UIBA)

I. Compute Error Locator Polynomial ($\rho = 0$) or Complementary Error-Erasure Evaluator Polynomial ($\rho > 0$)

- Inputs: $\mathbf{S} = [S_0, S_1, \dots, S_{2t-1}]$, $[\alpha_0, \alpha_1, \dots, \alpha_{\rho-1}]$
- Initialization: $\gamma^{(0)} = 1, L_B^{(0)} = L_\Lambda^{(0)} = 0$
 - $\rho > 0$: $\hat{\Omega}^{(0)}(x) = \hat{\Theta}^{(0)}(x) = S_0 + S_1x + \dots + S_{2t-1}x^{2t-1}$
 - $\rho = 0$: $\hat{\Theta}^{(0)}(x) = S_0 + S_1x + \dots + S_{2t-2}x^{2t-2} + x^{2t}$, $\hat{\Omega}^{(0)}(x) = S_0 + S_1x + \dots + S_{2t-2}x^{2t-2} + S_{2t-1}x^{2t-1} + x^{2t}$
- For $r = 0, 1, 2, \dots, 2t-1$, do:
 - Set $\Delta^{(r)} \leftarrow \begin{cases} \alpha_r, & \text{if } r < \rho \\ \hat{\Omega}_0^{(r)}, & \text{otherwise} \end{cases}$
 - If $r \leq \rho$, then set $\hat{\Theta}^{(r)}(x) \leftarrow \hat{\Omega}^{(r)}(x)$
 - $\hat{\Omega}^{(r+1)}(x) = \gamma^{(r)} \cdot [\mathcal{L}_1 \hat{\Omega}^{(r)}](x) - \Delta^{(r)} \cdot \hat{\Theta}^{(r)}(x)$
 - $\hat{\Theta}^{(r+1)}(x) \leftarrow \begin{cases} [\mathcal{L}_1 \hat{\Omega}^{(r)}](x), & \text{if } \Delta^{(r)} \neq 0 \& L_\Lambda^{(r)} \leq L_B^{(r)} \\ \hat{\Theta}^{(r)}(x), & \text{else} \end{cases}$
 - If $r < \rho$
 - * Set $\gamma^{(r+1)} \leftarrow \gamma^{(r)}$
 - * Set $L_B^{(r+1)} \leftarrow r+1, L_\Lambda^{(r+1)} \leftarrow r+1$
 - Else if $\Delta^{(r)} \neq 0 \& L_\Lambda^{(r)} \leq L_B^{(r)}$
 - * Set $\gamma^{(r+1)} \leftarrow \Delta^{(r)}$
 - * Set $L_\Lambda^{(r+1)} \leftarrow L_B^{(r)} + 1, L_B^{(r+1)} \leftarrow L_\Lambda^{(r)}$
 - Else
 - * Set $\gamma^{(r+1)} \leftarrow \gamma^{(r)}$
 - * Set $L_B^{(r+1)} \leftarrow L_B^{(r)} + 1, L_\Lambda^{(r+1)} \leftarrow L_\Lambda^{(r)}$
 - If $\rho = 0$, then set $\hat{\Theta}_{2t-r-2}^{(r+1)} \leftarrow 0$
 - If $\rho > 0 \& L_B^{(r+1)} = t + \lceil \frac{\rho}{2} \rceil$, then break For-loop
- Outputs: $L_\Lambda^{(2t)}, \hat{\Omega} = [\hat{\Omega}_0^{(2t)}, \hat{\Omega}_1^{(2t)}, \dots, \hat{\Omega}_{2t-1}^{(2t)}], \rho > 0$: $\Delta = [\alpha_0, \alpha_1, \dots, \alpha_{\rho-1}, \Delta^{(\rho)}, \Delta^{(\rho+1)}, \dots, \Delta^{(2t-1)}]$
 $\rho = 0$: $\mathbf{B} = [\hat{\Theta}_0^{(2t)}, \hat{\Theta}_1^{(2t)}, \dots, \hat{\Theta}_{2t-1}^{(2t)}, \gamma^{(2t)}]$ (return)

II. Compute Error-Erasure Locator Polynomial

- Input: $\Delta = [\alpha_0, \alpha_1, \dots, \alpha_{\rho-1}, \Delta^{(\rho)}, \Delta^{(\rho+1)}, \dots, \Delta^{(2t-1)}]$
- Initialization: $\hat{\Omega}^{(0)}(x) = \hat{\Theta}^{(0)}(x) = x^{2t}$, $\gamma^{(0)} = 1$, $L_B^{(0)} = L_\Lambda^{(0)} = 0$
- For $r = 0, 1, 2, \dots, 2t - 1$, do:
 - If $r \leq \rho$, then set $\hat{\Theta}^{(r)}(x) \leftarrow \hat{\Omega}^{(r)}(x)$
 - $\hat{\Omega}^{(r+1)}(x) = \gamma^{(r)} \cdot [\mathcal{L}_1 \hat{\Omega}^{(r)}](x) - \Delta^{(r)} \cdot \hat{\Theta}^{(r)}(x)$
 - $\hat{\Theta}^{(r+1)}(x) \leftarrow \begin{cases} [\mathcal{L}_1 \hat{\Omega}^{(r)}](x), & \text{if } \Delta^{(r)} \neq 0 \& L_\Lambda^{(r)} \leq L_B^{(r)} \\ \hat{\Theta}^{(r)}(x), & \text{else} \end{cases}$
 - If $r < \rho$
 - * Set $\gamma^{(r+1)} \leftarrow \gamma^{(r)}$
 - * Set $L_B^{(r+1)} \leftarrow r + 1, L_\Lambda^{(r+1)} \leftarrow r + 1$
 - Else if $\Delta^{(r)} \neq 0 \& L_\Lambda^{(r)} \leq L_B^{(r)}$
 - * Set $\gamma^{(r+1)} \leftarrow \Delta^{(r)}$
 - * Set $L_\Lambda^{(r+1)} \leftarrow L_B^{(r)} + 1, L_B^{(r+1)} \leftarrow L_\Lambda^{(r)}$
 - Else
 - * Set $\gamma^{(r+1)} \leftarrow \gamma^{(r)}$
 - * Set $L_B^{(r+1)} \leftarrow L_B^{(r)} + 1, L_\Lambda^{(r+1)} \leftarrow L_\Lambda^{(r)}$
 - If $\rho > 0 \& L_B^{(r+1)} = t + \lceil \frac{\rho}{2} \rceil$, then break For-loop
- Output: $\Lambda = [\hat{\Omega}_0^{(2t)}, \hat{\Omega}_1^{(2t)}, \hat{\Omega}_2^{(2t)}, \dots, \hat{\Omega}_{2t}^{(2t)}]$

Table II lists the hardware costs and the critical path delays among different architectures. In some sense, it is fair to claim that error-erasure decoding requires twice the complexity of error-only decoding. This is justified by, under serial implementation, **rIBA** exhibits twice the complexity of **eIBMA**; whereas under parallel implementation, **rPIBA** exhibits twice the complexity as **ePIBMA**. It is, however, counterintuitive that the parallel architecture **SPIBA** exploits 33% less hardware than the serial architecture **rIBA** while achieving more than twice faster throughput. The **SPIBA** architecture also compares favorably with the two-folded architecture (which trivially follows the methodology in [12]) of **rPIBA** wherein three latches and a multiplexer are added to each cell. The decoder architecture **EEIBA** employs the extended Euclidean algorithm to iteratively update error-erasure evaluator polynomial and produce discrepancy values, and in parallel performing the reformulated inversionless Blahut algorithm to iteratively update the error-erasure locator polynomial [22]. Its complexity is almost same as that of **rPIBA**, whereas its critical path exhibits an extra multiplex operation.

The proposed unified systolic architecture **UPIBA** is particularly attractive. It uses only half number of cells of a fully parallel implementation **rPIBA**, and exhibits virtually equal complexity as the error-only parallel decoder **ePIBMA**, while performing almost equally effective for error-only decoding as the dedicated decoder **ePIBMA**.

VII. DECODING ALGORITHMS AND ARCHITECTURES FOR SINGLY EXTENDED REED-SOLOMON CODES

A singly extended RS (SERS) $\mathcal{C}(n = 2^m, n - 2t, d = 2t + 1)$ code over $\text{GF}(2^m)$ is defined such that [15]

$$\mathcal{C} \triangleq \{ \mathbf{C} = [\mathbf{C}', C_{-1}] \in \text{GF}(2^m)^n : C'(\alpha^i) = 0, 1 \leq i < 2t, C_{-1} = C'(\alpha^{2t}) \}. \quad (25)$$

Recall that $\mathbf{C}' = [C'_{n-2}, C'_{n-3}, \dots, C'_1, C'_0]$ is systematically encoded by LFSR circuit and thus yields descending indexing. Therefore, it is natural to index the extended symbol by -1 . It is easy to verify that $\mathcal{C}$ has minimum nonzero weight $2t + 1$. If $C'(x) \neq 0$ is divisible by $\prod_{i=1}^{2t} (x - \alpha^i)$, then $\mathbf{C}'$ has the minimum weight $2t + 1$, so does $\mathbf{C}$; otherwise, $\mathbf{C}'$ has the minimum weight $2t$ and by assumption $C_{-1} = C'(\alpha^{2t}) \neq 0$, thus $\mathbf{C}$ has the minimum weight $2t + 1$.

A. Error-Only Decoder With ePIBMA

In this section we demonstrate that the proposed ePIBMA can be slightly modified to decode SERS codes. We first describe the complete decoding algorithm that deploys ePIBMA and justify it thereafter.

Error-Only Decoder With ePIBMA

I. Compute Syndromes

- Input: $\mathbf{R} = [\mathbf{R}', R_{-1}]$
- Compute $S_i = R'(\alpha^{i+1})$, $0 \leq i < 2t - 1$, $S_{2t-1} = R'(\alpha^{2t}) - R_{-1}$

II. Compute Error Locator Polynomial

- Initialization: $\hat{\Theta}^{(0)}(x) = S_0 + S_1x + \dots + S_{2t-2}x^{2t-2} + x^{2t}$, $\hat{\Omega}^{(0)}(x) = S_0 + S_1x + \dots + S_{2t-2}x^{2t-2} + S_{2t-1}x^{2t-1} + x^{2t}$, $\gamma^{(0)} = 1, L_\Lambda^{(0)} = L_B^{(0)} = 0, z = 1$
- For $r = 0, 1, 2, \dots, 2t - 1$, do:
 - if $r = 2t - 1 \& L_\Lambda^{(r)} < t$, then
 - set $\Delta^{(r)} \leftarrow 0, E_{-1} \leftarrow \hat{\Omega}_0^{(r)}$;
 - else set $\Delta^{(r)} \leftarrow \hat{\Omega}_0^{(r)}, E_{-1} \leftarrow 0$ (new)
 - $\hat{\Omega}^{(r+1)}(x) = \gamma^{(r)} \cdot [\mathcal{L}_1 \hat{\Omega}^{(r)}](x) - \Delta^{(r)} \cdot \hat{\Theta}^{(r)}(x)$
 - If $\Delta^{(r)} \neq 0$ and $L_\Lambda^{(r)} \leq L_B^{(r)}$, then
 - * Set $\hat{\Theta}^{(r+1)}(x) \leftarrow [\mathcal{L}_1 \hat{\Omega}^{(r)}](x)$
 - * Set $L_\Lambda^{(r+1)} \leftarrow L_B^{(r)} + 1, L_B^{(r+1)} \leftarrow L_\Lambda^{(r)}$
 - * Set $\gamma^{(r+1)} \leftarrow \Delta^{(r)}, z \leftarrow z \cdot \alpha^{-1}$
 - Else
 - * Set $\gamma^{(r+1)} \leftarrow \gamma^{(r)}, L_\Lambda^{(r+1)} \leftarrow L_\Lambda^{(r)}$
 - * If $L_B^{(r)} = t - 1$, then set $\hat{\Theta}^{(r+1)}(x) \leftarrow [\mathcal{L}_1 \hat{\Theta}^{(r)}](x), L_B^{(r+1)} \leftarrow L_B^{(r)}$
 - * Else set $\hat{\Theta}^{(r+1)}(x) \leftarrow \hat{\Theta}^{(r)}(x), z \leftarrow \alpha^{-1}z, L_B^{(r+1)} \leftarrow L_B^{(r)} + 1$
 - Set $\hat{\Theta}_{2t-r-2}^{(r+1)} = 0$

III. Correct Errors

- Run **eCSEE** to correct $\mathbf{R}' \rightarrow \mathbf{C}'$ with inputs $\{[\hat{\Omega}_0^{(2t)}, \hat{\Omega}_1^{(2t)}, \dots, \hat{\Omega}_t^{(2t)}], [\hat{\Theta}_0^{(2t)}, \hat{\Theta}_1^{(2t)}, \dots, \hat{\Theta}_{t-1}^{(2t)}], \gamma^{(2t)}, L_\Lambda^{(2t)}, z\}$
- Set $C_{-1} \leftarrow R_{-1} + E_{-1}\gamma\Lambda_0^{-1}$

TABLE III
SERS-DECODER COMPARISONS OF HARDWARE COMPLEXITY AND CRITICAL PATH

Architectures	Error Type	Adders	Multipliers	Latches	Muxes	Clocks	$T_{\text{crit-path}}$
eIBMA	Error-Only	$2t$	$3t + 2$	$3t + 3$	$t + 2$	$2t$	$2T_{\text{mult}} + \lceil \log_2(2t + 2) \rceil T_{\text{add}} + T_{\text{mux}}$
RiBMA [11]	Error-Only	$3t + 1$	$6t + 2$	$6t + 2$	$3t + 2$	$2t$	$T_{\text{mult}} + T_{\text{add}} + T_{\text{mux}}$
ePIBMA	Error-Only	$2t + 1$	$4t + 2$	$4t + 2$	$6t + 4$	$4t$	$T_{\text{mult}} + T_{\text{add}} + T_{\text{mux}}$
SPIBA	Error-Erasure	$2t + 1$	$4t + 2$	$4t + 2$	$4t + 3$	$4t$	$T_{\text{mult}} + T_{\text{add}} + 2T_{\text{mux}}$
UPIBA	Unified	$2t + 1$	$4t + 2$	$4t + 2$	$6t + 4$	$4t/2t$	$T_{\text{mult}} + T_{\text{add}} + 2T_{\text{mux}}$

If the length $L_{\Lambda}^{(2t-1)} < t$, then the discrepancy $\Delta^{(2t-1)}$ must be zero, otherwise, the resulting $\Lambda^{(2t)}(x)$ must have length $L_{\Lambda}^{(2t)} > t$ and thus is uncorrectable. The condition $L_{\Lambda}^{(2t-1)} < t$ further indicates that a nonzero discrepancy $\hat{\Omega}_0^{(2t-1)} \neq 0$ is resulted from the corruption of R_{-1} by an error E_{-1} . We now establish the relation between $\hat{\Omega}_0^{(2t-1)}$ and E_{-1} as follows. Let S_{2t-1}^* be the genuine syndrome associated with the spectrum α^{2t} , then, it holds

$$S_{2t-1}^* = R'(\alpha^{2t}) - (R_{-1} - E_{-1}) = S_{2t-1} + E_{-1}.$$

Recall that $\Delta^{(2t-1)} = 0$ implies

$$0 = \Lambda_0^{(2t-1)} S_{2t-1}^* + \sum_{i=1}^{L_{\Lambda}} \Lambda_i^{(2t-1)} S_{2t-1-i},$$

and $\hat{\Omega}_0^{(2t-1)}$ is due to

$$\hat{\Omega}_0^{(2t-1)} = \sum_{i=0}^{L_{\Lambda}} \Lambda_i^{(2t-1)} S_{2t-1-i}.$$

Inspecting the above two equalities yields

$$-\hat{\Omega}_0^{(2t-1)} = \Lambda_0^{(2t-1)} E_{-1}.$$

Since $\Lambda_0^{(2t-1)}$ is not an ultimate output and satisfies

$$\Lambda_0^{(2t-1)} = \left(\gamma^{(2t)} \right)^{-1} \Lambda_0^{(2t)},$$

we obtain

$$E_{-1} = -\hat{\Omega}_0^{(2t-1)} \gamma^{(2t)} \left(\Lambda_0^{(2t)} \right)^{-1}. \quad (26)$$

On the other hand, if $L_{\Lambda}^{(2t-1)} = t$, then $\mathbf{R}'$ is correctable only if R_{-1} , accordingly S_{2t-1} , is error free. Note E_{-1} is computed inside the Control unit using one multiplexer, and does not stay on the critical path.

In summary, it incurs virtually no cost to extend **ePIBMA** to decode SERS codes, except that a multiplex operation (due to choosing discrepancy $\Delta^{(r)}$) is added to the critical path of **ePIBMA**, yielding

$$T_{\text{crit-path}} = T_{\text{mult}} + T_{\text{add}} + T_{\text{mux}}. \quad (27)$$

Finally, we remark that **RiBMA** ([11]) may also be modified in the same manner to decode SERS codes, but its complexity substantially larger than that of **ePIBMA**, as listed in Table III.

B. Error-Erasure Decoder With SPIBA

Assume there are ρ erasure locators $\alpha_0, \alpha_1, \dots, \alpha_{\rho-1}$, respectively. We next investigate the error-erasure decoding conditioned on whether or not the extended symbol is erased. The first case is readily decoded by applying the same modification in **ePIBMA** to **SPIBA**. Specifically, the following lines are added to **SPIBA**.

- if $r = 2t - 1$ & $2L_{\Lambda} + \rho \leq 2t - 2$, then set $\Delta^{(r)} \leftarrow 0$, $E_{-1} \leftarrow \hat{\Omega}_0^{(r)}$;
else set $\Delta^{(r)} \leftarrow \hat{\Omega}_0^{(r)}$, $E_{-1} \leftarrow 0$ (new)

We now show the above method does not apply for the second case. Assume the number of errors, f , and the number of erasures, ρ , satisfy $\rho < 2t$ and $2f + \rho = 2t$, then the product of error locator polynomial and erasure locator polynomial requests $2t$ (consecutive) syndromes to determine, however, we have only $2t - 1$ valid syndromes. This is because the extended symbol is treated as both erasure locator and erroneous syndrome. A proper method is to treat the erased extended symbol as a syndrome error, but not an erasure locator. Consequently, the remaining $\rho - 1$ erasures are correctable by the $2t - 1$ valid syndromes if and only if $2f + (\rho - 1) \leq 2t - 1$, this is because $\rho - 1$ erasures and f errors occur within the RS subcode. Note this does not compromise the error correct capability, by taking into account the erased extended symbol. For notation simplicity, if the extended locator α^{-1} is erased, then it is always represented by the last erasure, i.e., $\alpha_{\rho-1} = \alpha^{-1}$. It is worth noting that the dynamic power saving mechanism in **SPIBA**,

- If $L_B^{(r+1)} = t + \lceil \frac{\rho}{2} \rceil$, then break For-loop

is invalid herein, since S_{2t-1} may be erroneous. Now we are ready to incorporate minor modifications into **SPIBA** for the error-erasure decoding of SERS codes.

Error-Erasure Decoder With SPIBA

I. Compute Syndromes

- Inputs: $\mathbf{R} = [\mathbf{R}', R_{-1}]$, $[\alpha_0, \alpha_1, \dots, \alpha_{\rho-1}]$
- If $\alpha_{\rho-1} = \alpha^{-1}$, then set $\rho \leftarrow \rho - 1$, $\varepsilon \leftarrow 1$; else $\varepsilon \leftarrow 0$
- Compute $S_i = R'(\alpha^{i+1})$, $0 \leq i < 2t - 1$, $S_{2t-1} = R'(\alpha^{2t}) - R_{-1}$
- Outputs: $[S_0, S_1, S_2, \dots, S_{2t-1}]$, ε , ρ

II. Compute Complementary Error-Erasure Evaluator Polynomial

- Inputs: $[S_0, S_1, \dots, S_{2t-1}]$, $[\alpha_0, \alpha_1, \dots, \alpha_{\rho-1}]$, ε , ρ
- Initialization: $\hat{\Omega}^{(0)}(x) = \hat{\Theta}^{(0)}(x) = S_0 + S_1x + \dots + S_{2t-1}x^{2t-1}$, $\gamma^{(0)} = 1$, $L_{\Lambda}^{(0)} = L_B^{(0)} = 0$

- For $r = 0, 1, 2, \dots, 2t - 1$, do:
 - Set $\Delta^{(r)} \leftarrow \begin{cases} \alpha_r, & \text{if } r < \rho \\ 0, & \text{if } r = 2t - 1 \text{ \& } 2L_\Lambda + \rho \leq 2t - 2 + \varepsilon \\ \hat{\Omega}_0^{(r)}, & \text{otherwise} \end{cases}$
 - If $r = 2t - 1$, then set $E_{-1} \leftarrow \begin{cases} \hat{\Omega}_0^{(r)}, & \text{if } 2L_\Lambda + \rho \leq 2t - 2 + \varepsilon \\ 0, & \text{otherwise} \end{cases}$
 - If $r \leq \rho$, then set $\hat{\Theta}^{(r)}(x) \leftarrow \hat{\Omega}^{(r)}(x)$
 - $\hat{\Omega}^{(r+1)}(x) = \gamma^{(r)} \cdot [\mathcal{L}_1 \hat{\Omega}^{(r)}](x) - \Delta^{(r)} \cdot \hat{\Theta}^{(r)}(x)$
 - $\hat{\Theta}^{(r+1)}(x) \leftarrow \begin{cases} [\mathcal{L}_1 \hat{\Omega}^{(r)}](x), & \text{if } \Delta^{(r)} \neq 0 \text{ \& } L_\Lambda^{(r)} \leq L_B^{(r)} \\ \hat{\Theta}^{(r)}(x), & \text{else} \end{cases}$
 - If $r < \rho$
 - * Set $\gamma^{(r+1)} \leftarrow \gamma^{(r)}$
 - * Set $L_B^{(r+1)} \leftarrow r + 1, L_\Lambda^{(r+1)} \leftarrow r + 1$
 - Else if $\Delta^{(r)} \neq 0 \text{ \& } L_\Lambda^{(r)} \leq L_B^{(r)}$
 - * Set $\gamma^{(r+1)} \leftarrow \Delta^{(r)}$
 - * Set $L_\Lambda^{(r+1)} \leftarrow L_B^{(r)} + 1, L_B^{(r+1)} \leftarrow L_\Lambda^{(r)}$
 - Else
 - * Set $\gamma^{(r+1)} \leftarrow \gamma^{(r)}$
 - * Set $L_B^{(r+1)} \leftarrow L_B^{(r)} + 1, L_\Lambda^{(r+1)} \leftarrow L_\Lambda^{(r)}$
- Outputs: $\hat{\Omega} = [\hat{\Omega}_0, \hat{\Omega}_1, \dots, \hat{\Omega}_{2t-1}]$, $\Delta = [\alpha_0, \alpha_1, \dots, \alpha_{\rho-1}, \Delta^{(\rho)}, \Delta^{(\rho+1)}, \dots, \Delta^{(2t-1)}]$, $\gamma^{(2t)}, L_\Lambda^{(2t)}, E_{-1}$

III. Compute Error-Erasure Locator Polynomial

- Inputs: $\Delta = [\alpha_0, \dots, \alpha_{\rho-1}, \Delta^{(\rho)}, \Delta^{(\rho+1)}, \dots, \Delta^{(2t-1)}]$, ρ
- Initialization: $\hat{\Omega}^{(0)}(x) = \hat{\Theta}^{(0)}(x) = x^{2t}$, $\gamma^{(0)} = 1$, $L_B^{(0)} = L_\Lambda^{(0)} = 0$
- For $r = 0, 1, 2, \dots, 2t - 1$, do:
 - If $r \leq \rho$, then set $\hat{\Theta}^{(r)}(x) \leftarrow \hat{\Omega}^{(r)}(x)$
 - $\hat{\Omega}^{(r+1)}(x) = \gamma^{(r)} \cdot [\mathcal{L}_1 \hat{\Omega}^{(r)}](x) - \Delta^{(r)} \cdot \hat{\Theta}^{(r)}(x)$
 - $\hat{\Theta}^{(r+1)}(x) \leftarrow \begin{cases} [\mathcal{L}_1 \hat{\Omega}^{(r)}](x), & \text{if } \Delta^{(r)} \neq 0 \text{ \& } L_\Lambda^{(r)} \leq L_B^{(r)} \\ \hat{\Theta}^{(r)}(x), & \text{else} \end{cases}$
 - If $r < \rho$
 - * Set $\gamma^{(r+1)} \leftarrow \gamma^{(r)}$
 - * Set $L_B^{(r+1)} \leftarrow r + 1, L_\Lambda^{(r+1)} \leftarrow r + 1$
 - Else if $\Delta^{(r)} \neq 0 \text{ \& } L_\Lambda^{(r)} \leq L_B^{(r)}$
 - * Set $\gamma^{(r+1)} \leftarrow \Delta^{(r)}$
 - * Set $L_\Lambda^{(r+1)} \leftarrow L_B^{(r)} + 1, L_B^{(r+1)} \leftarrow L_\Lambda^{(r)}$
 - Else
 - * Set $\gamma^{(r+1)} \leftarrow \gamma^{(r)}$
 - * Set $L_B^{(r+1)} \leftarrow L_B^{(r)} + 1, L_\Lambda^{(r+1)} \leftarrow L_\Lambda^{(r)}$
- Output: $\Lambda = [\hat{\Omega}_0, \hat{\Omega}_1, \hat{\Omega}_2, \dots, \hat{\Omega}_{2t}]$

IV. Correct Errors

- Apply the Chien search and modified Forney's formula (22) to correct $\mathbf{R}' \rightarrow \mathbf{C}'$ with inputs $\{\hat{\Omega}, \Lambda, L_\Lambda\}$
- Set $C_{-1} \leftarrow R_{-1} + E_{-1} \gamma \Lambda_0^{-1}$

Clearly, the error-erasure decoder architecture of SERS codes is literally same as that of unextended RS codes, except that a multiplex operation is added to the critical path of **SPIBA**, yielding

$$T_{\text{crit-path}} = T_{\text{mult}} + T_{\text{add}} + 2T_{\text{mux}}. \quad (28)$$

This is because two multiplexes must be sequentially carried to choose discrepancy $\Delta^{(r)}$ during computing the complementary error-erasure evaluator polynomial. Finally, it is trivial to extend UPIBA for a unified decoding of SERS codes while retaining the efficiency of both error-only and error-erasure decoding. Table III draws comparisons on different decoders. To the author's best knowledge, the modified B-M algorithm in [15, p. 216] is the only valid decoding algorithm for SERS codes in literature. In conclusion, all proposed high-speed decoder architectures in preceding sections are handily applicable to SERS codes with minor modifications.

VIII. CONCLUSION

In this paper, we devised new scalable Reed–Solomon decoder architectures. On error-only decoding, we devised an Enhanced Parallel Inversionless Berlekamp–Massey Algorithm (ePIBMA), through algorithmic transformation toward the inversionless Berlekamp–Massey algorithm. The outputs of ePIBMA are readily exploited by the generalized Horiguchi–Koetter formula for error evaluation, which avoids extra computation for the error evaluator polynomial while exhibiting similar complexity as the conventional Forney formula. The dedicated systolic architecture **ePIBMA** achieves the high clock frequency with one multiplication and one addition, while requiring only $4t + 2$ multipliers and registers (wherein t denotes the error correction capability), which stand favorably against existing high-throughput decoder architectures which use at least $6t$ multipliers and registers. On error-erasure decoding, we devised a Split Parallel Inversionless Blahut Algorithm (SPIBA), through algorithmic transformation toward the inversionless Blahut algorithm. Its dedicated systolic architecture **SPIBA** exhibits a short critical path with one multiplication, one addition and one multiplex, and uses only $4t + 2$ multipliers and registers. It employs 33% less hardware, at the same time achieves more than twice faster throughput, than the serial architecture directly derived from the inversionless Blahut algorithm. We further devised a Unified Parallel Inversionless Blahut Algorithm (UPIBA), by merging **ePIBMA** and **SPIBA**. Its complexity is literally same as **SPIBA/ePIBMA** while achieving the efficiency of **ePIBMA** in error-only decoding and **SPIBA** in error-erasure decoding. We also demonstrated that all three architectures are readily deployed to decode singly extended Reed–Solomon codes with negligible add-on's. To the authors best knowledge, their high-throughput decoding is the first time explored.

All above three proposed architectures also exhibit the following desirable features. (i). they save power dynamically at the predominant scenario that the combined number of errors and half number of erasures is smaller than t . **ePIBMA** is capable of dynamically disabling the multipliers and adders

when the error-locator polynomial $\Lambda(x)$ is converged. **SPIBA** enables to terminate early when $\Lambda(x)$ is determined to converge. **UPIBA** inherits it from both **ePIBMA** and **SPIBA**. This feature is exclusive from all existing decoder architectures. (ii). They naturally execute the role of linear-feedback-shift-register (LFSR) encoder by using appropriate inputs, without circuit modification. This enables to eliminate the LFSR encoder when encoding and decoding are divided in time. (iii). Their output locations are invariant to the value of t . This is highly desirable for configurable design, a common-practice in industry. Finally, we remark that it is straightforward to pipeline the three architectures to achieve much lower latency in an interleaved environment following similar strategies in [7], [11].

ACKNOWLEDGMENT

The author would like to express deep appreciation to Prof. Ralf Koetter on his challenges and inspirations. The author also thanks the associate editor Khaled Abdel-Ghaffar for many detailed comments and anonymous reviewers for some insightful technical challenges that help improve the quality of the paper.

REFERENCES

- [1] E. R. Berlekamp, *Algebraic Coding Theory*. New York, NY, USA: McGraw-Hill, 1968.
- [2] Y. Sugiyama, M. Kasahara, S. Hirasawa, and T. Namekawa, "A method for solving key equation for decoding Goppa codes," *Inf. Control*, vol. 27, no. 1, pp. 87–99, Jan. 1975.
- [3] L. R. Welch and E. R. Berlekamp, "Error correction for algebraic block codes," U.S. Patent 4633 470, Dec. 30, 1986.
- [4] H. M. Shao, T. K. Truong, L. J. Deutsch, J. H. Yuen, and I. S. Reed, "A VLSI design of a pipeline Reed–Solomon decoder," *IEEE Trans. Comput.*, vol. 34, no. 5, pp. 393–403, May 1985.
- [5] E. R. Berlekamp, G. Seroussi, and P. Tong, "A hypersystolic Reed–Solomon decoder," in *Reed–Solomon Codes and Their Applications*, S. B. Wicker and V. K. Bhargava, Eds. Piscataway, NJ, USA: IEEE Press, 1994.
- [6] H.-C. Chang and C. B. Shung, "New serial architecture for the Berlekamp–Massey algorithm," *IEEE Trans. Commun.*, vol. 47, no. 4, pp. 481–483, Apr. 1999.
- [7] H. Lee, "High-speed VLSI architecture for parallel Reed–Solomon decoder," *IEEE Trans. VLSI Syst.*, vol. 11, no. 2, pp. 288–294, Apr. 2003.
- [8] J. H. Baek and M. H. Sunwoo, "New degree computationless modified Euclid algorithm and architecture for Reed–Solomon decoder," *IEEE Trans. VLSI Syst.*, vol. 14, no. 8, pp. 915–920, Aug. 2006.
- [9] J. H. Baek and M. H. Sunwoo, "Simplified degree computationless modified Euclid's algorithm and its architecture," in *Proc. IEEE ISCAS*, May 2007, pp. 905–908.
- [10] I. S. Reed, M. T. Shih, and T. K. Truong, "VLSI design of inverse-free Berlekamp–Massey algorithm," in *Proc. Inst. Elect. Eng.—E Comput. Digital Techn.*, Sep. 1991, vol. 138, pp. 295–298.
- [11] D. V. Sarwate and N. R. Shanbhag, "High-speed architectures for Reed–Solomon decoders," *IEEE Trans. VLSI Syst.*, vol. 9, no. 5, pp. 641–655, Oct. 2001.
- [12] K. Seth, K. N. Viswajith, S. Srinivasan, and V. Kamakoti, "Ultra folded high-speed architectures for Reed–Solomon decoders," in *Proc. Int. Conf. VLSI Des.*, Jan. 2006, pp. 517–520.
- [13] J.-L. Park and H. Lee, "Area-efficient truncated Berlekamp–Massey architecture for Reed–Solomon decoder," *Electron. Lett.*, vol. 47, no. 4, pp. 241–243, Feb. 2011.
- [14] G. D. Forney, Jr., "On decoding BCH codes," *IEEE Trans. Inf. Theory*, vol. 11, no. 4, pp. 549–557, Oct. 1965.
- [15] R. E. Blahut, *Algebraic Codes for Data Transmission*. Cambridge, U.K.: Cambridge Univ. Press, 2003.
- [16] K. G. Boyer, "Pipelined high speed Reed–Solomon error/erasure decoder," U.S. Patent 6 347 389, Feb. 12, 2002.
- [17] J.-H. Jeng and T.-K. Truong, "On decoding of both errors and erasures of a Reed–Solomon code using an inverse-free Berlekamp–Massey algorithm," *IEEE Trans. Commun.*, vol. 47, no. 10, pp. 1488–1494, Oct. 1999.
- [18] A. Kravtchenko, "Reed–Solomon Decoder," U.S. Patent 2003/022984, Dec. 11, 2003.
- [19] J.-S. Oh and K.-T. Oh, "Reed–Solomon decoder and decoding method utilizing a control signal indicating a new root for an initial error locator polynomial with respect to new erasure information," U.S. Patent 6 304 994, Oct. 16, 2001.
- [20] D. M. Banks, "Reed–Solomon decoder and decoding method for errors and erasures decoding," U.S. Patent 7 096 409, Aug. 22, 2006.
- [21] H.-C. Chang, C.-C. Lin, F.-K. Chang, and C.-Y. Lee, "A universal VLSI architecture for Reed–Solomon error-and-erasure decoders," *IEEE Trans. Circuits Syst.*, vol. 56, no. 9, pp. 1960–1967, Sep. 2009.
- [22] T.-K. Truong, J.-H. Jeng, and K.-C. Hung, "Decoding method for correcting both erasures and errors of Reed–Solomon codes," U.S. Patent 6 449 746, Sep. 10, 2002.
- [23] T.-K. Truong, J.-H. Jeng, and K.-C. Hung, "A new decoding algorithm for correcting both erasures and errors of Reed–Solomon codes," *IEEE Trans. Commun.*, vol. 51, no. 3, pp. 381–388, Mar. 2003.
- [24] R. T. Chien, "Cyclic decoding procedures for the Bose–Chaudhuri–Hocquenghem codes," *IEEE Trans. Inf. Theory*, vol. 4, no. 10, pp. 357–363, Oct. 1964.
- [25] "Digital transmission standard for cable television," American National Standard, Washington, DC, USA, ANSI/SCTE 07 2006, Jul. 2013.
- [26] Y.-F. Li, G.-C. Yang, H. Chen, and W. C. Kwong, "Extended Reed–Solomon codes for optical CDMA," *IEEE Trans. Commun.*, vol. 61, no. 6, pp. 2498–2507, Jun. 2013.
- [27] L. L. Joiner and J. J. Komo, "Time domain decoding of extended Reed–Solomon codes," in *Proc. IEEE Southeastcon Bringing Together Educ., Sci. Technol.*, 1996, pp. 238–241.
- [28] Y. Chen and K. K. Parhi, "Small area parallel Chien search architectures for long BCH codes," *IEEE Trans. VLSI Syst.*, vol. 12, no. 5, pp. 545–549, May 2004.
- [29] J. Cho and W. Sung, "Strength-reduced parallel Chien search architecture for strong BCH codes," *IEEE Trans. Circuits Syst.-II: Exp. Briefs*, vol. 55, no. 5, pp. 427–431, May 2008.



Yingquan Wu (M'07–SM'08) was born in China. He received the B.S. and M.S. degrees in mathematics from Harbin Institute of Technology, China, in July 1996 and July 1997, respectively; the M.S. degree in electrical engineering from the State University of New York at Buffalo in August 2000; and the Ph.D. degree in electrical and computer engineering from the University of Illinois at Urbana-Champaign in October 2004. He is currently a Cofounder and the Chief Scientist at Tidal Systems Inc., Santa Clara, CA, where he is responsible for algorithms and architectures for lossless compression, error control coding and digital signal processing for solid-state drive. His research interests include data compression, algebraic coding theory, soft-decision decoding techniques, and the efficient VLSI design of general communication algorithms.